

Highlights

The governor is part of the model: duty-cycle-aware deployment of object detectors on edge accelerators

Dinh Thuan Nguyen[✉], Lam Phuong Nguyen[✉], Vinh Huy Nguyen[✉], Sy Vu Quang, Mohan Rajesh Elara[✉], Anh Vu Le[✉]

- On Jetson, the stock GPU governor triples serial inference time.
- Host restructuring beats a maximum-clock serial pipeline by 2.4×.
- Sleeping accelerator waits trigger host down-clocking and lose 20% of sequential throughput.
- At camera rates, idle power dominates energy while scheduling sets latency.
- Pipeline gains transfer across neural accelerators, but mechanisms differ.

The governor is part of the model: duty-cycle-aware deployment of object detectors on edge accelerators

Dinh Thuan Nguyen^a, Lam Phuong Nguyen^a, Vinh Huy Nguyen^b, Sy Vu Quang^c, Mohan Rajesh Elara^d and Anh Vu Le^{a,*}

^aFaculty of Electrical and Electronics Engineering, Ton Duc Thang University, Ho Chi Minh City, Vietnam

^bCenter of Visualization and Simulation, Duy Tan University, Da Nang, Vietnam

^cAdvanced Intelligent Technology Research Group, Faculty of Electrical and Electronics Engineering, Ton Duc Thang University, 19 Nguyen Huu Tho Street, Tan Phong Ward, District 7, Ho Chi Minh City 700000, Vietnam

^dROAR Lab, Engineering Product Development, Singapore University of Technology and Design, Singapore

ARTICLE INFO

Keywords:

Edge AI
Object detection
DVFS
Embedded GPU
Energy efficiency
Jetson
Benchmarking methodology

ABSTRACT

The deployment cost of NMS-free detectors is equated with the latency of the compiled engine. We measure the full decode-to-detections pipeline of YOLO26n on a Jetson Orin Nano Super: module input power, clocks, GPU load and official COCO accuracy on 5,000 val2017 images. The same FP16 TensorRT engine that takes 4.2 ms back-to-back takes 13.2 ms inside a sequential application, because the stock GPU governor holds the GPU at minimum clock. A duty-cycle model describes the measurements: engine time scales with the inverse of the clock (3.7% mean absolute error, fitted, on a fixed-frequency sweep), interior-clock operating points cluster in an observed 0.53–0.74 GPU-load range, and a pipeline released from the top clock falls to the floor in seconds. Restructuring only the host pipeline, under default governors and with identical predictions, gives 2.4× the throughput of the best sequential pipeline at locked maximum clocks. The CPU governor is coupled to the same loop: a sleeping CUDA wait lets `schedutil` lower the host’s clock and, through slower submission, the GPU’s; a per-task utilization clamp recovers most of this loss at lower energy; on the full pipeline the wait policy is free (sleeping matches spinning under either governor). At camera rates below capacity, idle power sets energy per frame and the schedule sets latency. Replications on a Raspberry Pi 5 with Hailo-8 NPU and a Qualcomm QCM6490 show the effect transfers while its mechanism shifts to the host side. Edge-detection efficiency should be reported for the whole pipeline under deployment governors.

1. Introduction

Real-time object detection on embedded GPUs is usually evaluated with two numbers per model: accuracy on COCO [1] and a latency taken from an isolated benchmark of the compiled engine. Efficiency research then works mainly on the model. Structured pruning [2, 3], quantization-aware training [4] and distillation [5] are judged by how much engine latency they remove per unit of accuracy lost. Recent NMS-free detectors make this framing seem even more natural. YOLOv10 [6] and YOLO26 [7, 8] emit a fixed set of final detections from a one-to-one head, so the host only decodes the image and converts a few hundred output rows. With post-processing nearly gone, the engine benchmark should predict the application.

We find that on an NVIDIA Jetson Orin Nano Super it does not. The FP16 TensorRT engine of YOLO26n completes an inference in 4.2 ms when run back-to-back. Inside a conventional sequential application that decodes a JPEG, letterboxes it, runs the engine and builds detection rows, the same engine takes 13.2 ms. The model, input and builder are unchanged. What changes is the GPU’s duty cycle. While

the CPU decodes, the GPU idles, and the stock frequency governor responds by holding the GPU at its minimum clock of 306 MHz, 30% of its maximum. The application therefore never runs the model in the regime that the benchmark measures. This also changes how compression should be judged: a model that is 20% cheaper in the benchmark would gain little in such an application — at a fixed clock the saving is diluted by the host stages, and the clock itself does not move.

We did not set out to study governors. We first tried to make YOLO26n cheaper: object-aware structured pruning, quantization-aware training (QAT), distillation, compiler-guided pruning of the most expensive head layers, and a per-image adaptive-resolution router. These variants either lost several AP points or did not run faster once compiled, and the router failed verification on held-out images. We then left the model alone and restructured only the host pipeline. Under the same default governors and with byte-identical predictions, that change gave 2.4× the throughput of the best sequential pipeline even when the latter runs with all clocks locked at maximum, and 6.9× that of the standard Ultralytics predictor.

This paper is a systematic measurement study of that gap. It is guided by five research questions:

RQ1 How does the GPU governor’s response to an application’s duty cycle determine the in-application cost of a fixed engine, and can a simple model describe it?

Corresponding author.

E-mail addresses: nguyendinhthuan.st@tdtu.edu.vn (D. T. Nguyen),
nguyenphuonglam.st@tdtu.edu.vn (L. P. Nguyen),
nguyenvinhhuu@tdtu.edu.vn (V. H. Nguyen), vuquangsy@tdtu.edu.vn
(S. V. Quang), rajeshelara@sutd.edu.sg (M. R. Elara),
leanhvu@tdtu.edu.vn (A. V. Le).

- RQ2** Which host-pipeline decisions matter for throughput and energy, how large is each effect, and which of them interact with the governor?
- RQ3** How do these decisions trade latency against energy when frames arrive at camera rates rather than as a backlog?
- RQ4** How do model-level decisions (input resolution, compression, model size) rank once the pipeline is fixed?
- RQ5** Do the findings carry over to other NMS-free detectors and to other accelerator architectures?

Our contributions are:

1. A duty-cycle model of in-application engine time and energy (Section 3), checked against direct measurements (Section 5). Engine time scales with the inverse of the measured clock. Using the GPU load reported by the driver, we show that the measurements are consistent with a load-band governor whose observed band is schedule-specific, and that a sequential pipeline sits at or below the observed band’s top even at the minimum clock, which is consistent with why it never leaves the floor. A split of energy into an idle floor and a marginal term provides an empirical upper envelope for the measured below-capacity streaming cells (Section 7).
2. A controlled, cumulative pipeline ablation on all 5,000 COCO val2017 images (Section 6). Scheduling variants produce byte-identical predictions, so accuracy is held exactly constant. For each variant we report official COCO AP, throughput, input energy, GPU clock and CPU time, under both default and locked clocks, and we sweep the number of decode workers. Among other effects, we show that the choice between two CUDA synchronization calls costs 17–20% of throughput under the default governors. On the sequential and prefetch schedules this loss is governor-mediated and drops below 1% with locked clocks; on the full two-worker schedule the wait policy itself is free — a sleeping event wait matches a spinning one within 1% under either governor — and the 24% that stream synchronization still costs there is a completion-boundary effect, identical at locked clocks. A controlled test traces the governor-mediated loss to the CPU governor: a sleeping wait lets `schedutil` lower the clock of the host’s frequency domain. Keeping that domain busy recovers 97% of the loss, and a per-task utilization clamp [9], set without root privileges, recovers most of it at lower energy. We also show that garbage-collection pauses in a typical evaluation loop cost a fast pipeline 7.1% of its measured throughput.
3. A streaming evaluation at 15, 30 and 60 fps (Section 7). It shows that energy per frame is set mainly by idle power, that the configuration with the highest throughput adds one frame period of latency at low

rates, and that an arrival-aware variant removes this cost while keeping p95 latency at or below 41 ms at all three rates.

4. An evaluation of model-level decisions under a fixed, optimized pipeline (Section 8). Lower input resolution buys 27% throughput in a naive pipeline but only 2–4% in an optimized one (6–10% with Python’s cyclic garbage collector disabled). Our compression and routing attempts are reported as negative results. On YOLO26s and YOLOv10n, the pipeline effects are comparable in size to YOLO26n’s (5.6–6.9× over the standard predictor).
5. A replication of the core experiments on two further boards (Section 9): a Raspberry Pi 5 with a Hailo-8 NPU, whose accelerator has no user-visible frequency control, and a Rubik Pi 3 (Qualcomm QCM6490) measured twice, on its CPU and on its Hexagon NPU. The pipeline ladder transfers (3.3× on the Pi 5 with byte-identical predictions, 2.7× before the wait-policy change), while the Jetson-specific accelerator-governor signature is not directly observed on the NPU stacks, where the identifiable mechanisms shift to the host side; on CPU inference the ladder flattens and sustained load becomes thermally limited, and on the NPU stack the CPU governor becomes the largest term (57% on the matched overlap rungs), because the scheduler down-clocks the core that feeds the accelerator (from 2.7 GHz pinned to 0.95–1.8 GHz, depending on how little host work the schedule leaves on the critical path).

We do not propose a new detector, compression method or governor, and pipelining itself is standard practice. The contribution is the measurement and its explanation. On this class of device, the interaction between the host pipeline and the frequency governor determines more of the end-to-end cost than the model-level choices that usually receive the attention. We provide all scripts, raw power traces and prediction hashes, so that the results can be checked and reproduced (see the data and code availability statement).

2. Related Work

NMS-free detectors. Set-prediction detectors remove non-maximum suppression (NMS) by one-to-one matching [10]; RT-DETR made this regime real-time on GPUs [11]. YOLOv10 introduced consistent dual assignments so that a convolutional YOLO can be deployed with a one-to-one head and no NMS [6], and YOLO26 continues this design with an end-to-end head [7, 8]. With NMS gone, the remaining host work is decode, letterbox, and a short top-*k* row conversion. This makes the rest of the pipeline more visible, and it is why we study these models.

Compression for edge detection. Structured channel pruning [2, 3], quantization-aware training [4] and knowledge distillation [5] are the standard tools for making detectors cheaper. They are usually judged by parameter count,

FLOPs or isolated engine latency. We run these tools on the same deployment stack and report them in end-to-end units. Our negative results do not show that compression is ineffective in general. They show that, for a nano-scale NMS-free model on this device, the model is not where the time goes.

Latency prediction, DVFS and thermal control. nn-Meter showed that kernel-level effects make FLOPs a poor latency proxy on edge devices [12]. Tang et al. measured how GPU core and memory frequencies change the time and energy of deep learning workloads, and found that the effect differs across networks [13]. Nabavinejad et al. chose batch size and GPU frequency jointly for inference under latency limits [14]. Other DVFS work on mobile and edge processors learns frequency policies that avoid throttling [15], models GPU latency under DVFS [16], or controls frequency per operator on Jetson-class devices [17]. SparseDVFS also documents, as motivation, the antagonistic effect of the stock CPU and GPU governors on a Jetson Orin Nano: the CPU governor down-clocks the host during GPU-heavy phases, so the host is slow exactly when the GPU next requests data [17]. Section 6.3 measures the same coupling inside a detection pipeline and traces it to the CUDA wait policy. Work on 3D mobile games showed that managing CPU and GPU frequencies jointly saves power compared with independent governors, because the two processors depend on each other’s progress [18]. Closest to our setting, Karzhaubayeva et al. characterize CNN workloads on a Jetson TX2 and design integrated CPU–GPU DVFS governors for them [19], Lotus manages CPU and GPU frequencies jointly for object detectors on a Jetson Orin Nano to stabilize latency under thermal variation [20], and Kang shows that latency estimators for deadline-aware governors on Jetson Orin must condition on the memory clock [21]. All of these works set frequencies, or batch sizes, from outside the application — they design or evaluate frequency-control policies. We leave the stock governors in place and ask how an application’s duty cycle interacts with them: the schedule changes the operating state the stock governors observe, and thereby the measured cost of an otherwise fixed engine. We find a similar CPU–GPU governor coupling in a detection pipeline, and we test one standard kernel mechanism, utilization clamping [9], as a remedy that needed no root privileges on our system.

Characterization of edge inference. Mittal surveyed optimized deep learning on Jetson devices, covering precision, framework and power-mode choices [22]. Hadidi et al. compared frameworks and devices for edge inference and reported that, for the same model, the choice of framework changes inference time and energy considerably [23]. These studies measure the model, or the framework that runs it. We measure the application around it: we hold the engine fixed and change only host code and governor settings.

End-to-end costs outside the model. The “AI tax” study showed that pre- and post-processing can dominate accelerated AI applications [24]. On Jetson, JPEG decode and preprocessing have been offloaded

Table 1

Positioning against the closest measurement studies. “Freq.” is how frequency enters the experiment: set directly by the experimenter, replaced by a newly designed governor (“new gov.”), or left to the stock governor and observed (“modif.”: a modified stock governor; “ucl.”: per-task utilization clamping). “Pipe.”: the whole decode-to-detections pipeline is measured, not only the model. “Acc.”: accuracy is held constant or verified per variant. “Stream.”: camera-rate arrivals are measured, not only backlog throughput. SLO: service-level objective. A dash means the work does not address the aspect.

Work	Workload	Freq.	Pipe.	Acc.	Stream.
Tang et al. [13]	DNN time/energy	direct	—	—	—
Nabavinejad et al. [14]	batching w/ SLOs	direct	—	—	—
SparseDVFS [17]	per-operator DVFS	direct	—	yes	—
Pathania et al. [18]	3D mobile games	modif.	partial	—	—
Karzhaubayeva [19]	CNNs on TX2	new gov.	—	—	—
Lotus [20]	detec., Orin Nano	new gov.	—	—	—
Kang [21]	gov. latency est.	direct	—	—	—
AI tax [24]	pre/post-processing	—	yes	—	—
R-TOD [29]	end-to-end delay	—	yes	yes	yes
Stage-wise latency [27]	YOLO on Orin NX	stock	yes	—	—
This work	detection, 3 boards	stock+ucl.	yes	yes	yes

to dedicated units [25], training pipelines have been characterized for I/O pipelining and power modes [26], and the latency distribution of a YOLO pipeline on Jetson Orin NX has been modeled stage by stage as a function of image resolution [27]. MLPerf Inference standardizes measurement scenarios, separating offline throughput from server and single-stream latency [28]. For autonomous-driving detection, R-TOD analyzes and minimizes the end-to-end delay from frame exposure to reported detections by restructuring capture and pipelining (on-demand capture, zero-slack and contention-free pipelines) without changing the network [29]; our arrival-aware variant (Section 7) is a simple member of the same family, and we measure its interaction with the stock governors and the idle floor rather than worst-case delay. We do not claim that pipelining or preprocessing bottlenecks are new. What we add is a measured account, on one device and stack, of how the governor’s response to the application duty cycle changes the apparent cost of the *model*. We also show how this reorders deployment decisions such as compression and resolution.

Adaptive resolution. Choosing input scale per image or frame can save compute in detection [30, 31]. Our frozen pre-inference router is a simple member of this family. We report it because it failed held-out verification, and because its expected saving is smaller than the pipeline effects we measure.

Table 1 summarizes the positioning. The gap we fill is not pipelining, preprocessing or DVFS per se, but the interaction with the *stock* governors under the application’s own schedule and arrival process, with accuracy held exactly constant so that the timing effects are attributable.

3. A duty-cycle model of the detection pipeline

We use a deliberately simple model to organize the measurements. It has three parts. The first describes how engine time depends on the GPU clock. The second describes how each pipeline schedule determines the GPU’s duty cycle. The third describes how the governor picks a clock for that duty cycle. A fourth relation splits energy into an idle floor and a marginal part. Each part is checked against measurements in Sections 5 to 7.

Pipeline stages. For each image the application performs a decode-and-letterbox stage D on the CPU, an optional host normalization N (float-input engines only), an engine execution G on the GPU (including copies, which take under 1 ms on this unified-memory device), and a row construction stage R that converts the 300 output rows to image coordinates. Let M denote the remaining per-image work of the main thread (buffer fill, enqueue, synchronization).

Engine time. For a fixed engine, we model the GPU time at clock f as

$$G(f) = G_{\max} \frac{f_{\max}}{f}, \quad (1)$$

where $G_{\max}$ is the time at the maximum clock $f_{\max}$. This assumes the engine is limited by the GPU clock and not by memory. For variable-clock runs we use the time-weighted harmonic mean of the sampled clock as an approximate summary consistent with Eq. (1).

Schedules and duty cycle. The period T between completed images, and therefore throughput $X = 1/T$, follows from which stages lie on the critical path. For k decode workers:

$$T_{\text{seq}} = D + N + G(f) + R + M, \quad (2)$$

$$T_{\text{prefetch}} \approx \max\left(\frac{D+N}{k}, G(f) + R + M\right), \quad (3)$$

$$T_{\text{overlap}} \approx \max\left(\frac{D+N}{k}, R + M, G(f)\right). \quad (4)$$

The GPU busy fraction is $u = G(f)/T$. In the sequential schedule every host millisecond is GPU idle time. Eq. (3) describes the prefetch transition: host stages leave the critical path, and the period becomes the maximum of the worker rate and the GPU-serial part. The stage times in it are clock-dependent, so we use it as a structure rather than a fitted predictor (Section 5 measures them at a down-clocked host; at the prefetch rung’s higher clocks the decode rate is correspondingly higher). Overlap also removes R (Eq. (4)).

Governor. The GPU governor (nvhost_podgov, whose devfreq sysfs node on our device exposes only a 25 ms polling interval and no load thresholds) ships in NVIDIA’s public Jetson Linux kernel sources [32]. We did not attempt to derive its thresholds from the source and instead characterize it empirically as a load-band controller. The driver exposes the load it acts on through sysfs, which lets us check the busy fraction u directly (Section 5). It raises f while u exceeds an upper threshold u_{hi} , lowers it while u is below a lower threshold u_{lo} , and is clamped to $[f_{\min}, f_{\max}]$. A schedule therefore settles either at an interior clock with

$u \in [u_{\text{lo}}, u_{\text{hi}}]$ or at one of the clamps. This view implies a fixed point that we observe. In Eq. (2), u increases as f decreases, because a slower GPU occupies a larger share of the period. If, even at $f_{\min}$, the sequential pipeline’s busy fraction $G(f_{\min})/T_{\text{seq}}(f_{\min})$ stays below u_{hi} , the governor never has a reason to raise the clock. The application then runs at the minimum clock indefinitely, even though the GPU is busy about half of the time (measured loads 0.46–0.66 at the floor, Section 5).

Energy. Let P_{idle} be the module input power with no workload under a given clock policy, and P_{board} the mean power during a run. Energy per image splits as

$$E = \frac{P_{\text{board}}}{X} = \underbrace{\frac{P_{\text{idle}}}{X}}_{\text{idle reference}} + \underbrace{\frac{P_{\text{board}} - P_{\text{idle}}}{X}}_{E_{\text{marg}}}. \quad (5)$$

The first term is an accounting reference — the idle draw of the board in the measured power state, amortized over throughput — not a universal lower bound: a different sleep or power-gating regime could go below it. On a backlog, a faster schedule shrinks the idle-reference term. For a camera that delivers frames at a fixed rate r below the pipeline’s capacity, the throughput is $X = r$ regardless of schedule, so

$$E(r) \approx \frac{P_{\text{idle}}}{r} + E_{\text{marg}}. \quad (6)$$

The schedule then affects energy only through E_{marg} . The model therefore implies that pipeline optimization yields large energy savings on a backlog, but only small ones at fixed camera rates. Section 7 tests this prediction and finds that Eq. (6) with the backlog E_{marg} substituted is an empirical upper envelope for the below-capacity cells, not an exact prediction.

4. Experimental methodology

4.1. Platform and software

Table 2 lists the platform. Throughout the paper, “default” means the stock governors: nvhost_podgov for the GPU (eight steps from 306 to 1020 MHz) and schedutil for the CPU. “Locked” means jetson_clocks, which pins the CPU, GPU and memory controller (EMC) clocks to their maxima [33]; we verified the EMC pin by readback at ≈ 4 Hz, idle and under a 15 s inference load (constant 3199 MHz). On this release it leaves the fan under its own thermal control unless asked otherwise, and we did not ask. Each locked phase stores the default configuration in a fresh file, applies the lock, and restores the stored configuration afterwards. We read back the governor and clock limits after every phase. The two NumPy versions in Table 2 differ because Ultralytics 8.4.172 requires the newer release; the difference touches only the short COCO row-conversion step, whose code is identical across our pipelines and the baseline.

Table 2

Platform and software. BSP is the board support package.

Board	Jetson Orin Nano Super Developer Kit, 8 GB module
CPU	6x Cortex-A78AE, two frequency domains (CPUs 0–3, 4–5), 115–1728 MHz
GPU	Ampere, 1024 cores, 306–1020 MHz
Power mode	MAXN_SUPER
OS / BSP	Ubuntu 22.04, Jetson Linux R36.5.2
Kernel	5.15.199-tegra
Cooling	fan under thermal control (nvfancontrol); locked runs pin CPU, GPU and EMC clocks, not the fan
CUDA / TensorRT	12.6 / 10.3.0
Detector software	Ultralytics 8.4.172
Baseline runtime	PyTorch 2.5.0 (NVIDIA JetPack build)
Host libraries	Python 3.10.12, OpenCV 4.10.0, NumPy 1.21.5 (ours), 1.26.4 (Ultralytics)
Power sensor	INA3221 VDD_IN (module input), 20 Hz

4.2. Replication platforms

Two further boards (Section 9) rerun the core protocol with the smallest changes their stacks require. On the Raspberry Pi 5 the model is the Hailo Model Zoo’s precompiled HEF (Hailo Executable Format) `yolo8n_h8.hef` [34] for the Hailo-8 (INT8, NMS on chip, score threshold 0.2 fixed at compile time), driven through HailoRT 4.23.0 asynchronous pipelines with the same prefetch/overlap/spin switches as our TensorRT pipeline. The replication tables use short rung names: `seq` is the sequential pipeline, `prefetch` adds decode prefetch, `full w1/w2/w3` add post-processing overlap with one, two or three decode workers, and `spin` replaces the sleeping wait; power comes from the Pi 5’s power-management IC (PMIC) rail sum, which covers the system-on-chip (SoC) but not the 5 V loads (the Hailo-8 itself, the fan, USB), so Pi energies are SoC-side lower bounds. The Rubik Pi 3 (Ubuntu 24.04.4, kernel 6.8) is measured with two stacks. The CPU stack is ONNX Runtime 1.30.0 (default thread settings) [35] on the same YOLO26n ONNX graph as the Jetson float engine. The NPU stack runs an INT8 QNN context binary of the same graph (standard (1,84,8400) head, host NMS at the val settings) on the Hexagon HTP: the board’s QNN 2.46 predates the QAI RT 2.49.40-built context, so we drive it through the `qai-appbuilder 2.50.40` wheel’s bundled runtime. Because that userspace is not robust under concurrent calls, all NPU inference is serialized on one thread and host stages overlap around it. The board offers no trustworthy power sensor, so the Rubik sampler logs only the three CPU frequency domains and the SoC thermal zone; it does not sustain 20 Hz, and its effective rate is 13–18 Hz. The Pi sampler reads the PMIC’s SoC-side power rails at an effective 10 Hz. Energy integrals are trapezoids over the actual sample timestamps. The HTP stack is bit-deterministic (hash-verified like the Jetson and Pi); the multi-threaded CPU stack emits rows in a nondeterministic order, so there we verify equivalence on canonicalized (sorted) prediction rows and by COCOeval on each rung’s saved predictions.

4.3. Models and engines

The main model is the pretrained YOLO26n checkpoint [7] (2.57 M parameters). For generality we use YOLO26s (10.0 M) and YOLOv10n (2.78 M) [6]. All are exported to ONNX with `nms=False`, which selects the one-to-one head and yields a (1, 300, 6) tensor of boxes, scores and class indices with no NMS. Ultralytics 8.4.172 loads the YOLO26 head with the one-to-many branch by default. We therefore verified that `nms=False` selects the end-to-end branch in both export and prediction. Every engine is a static batch-1 FP16 TensorRT engine built on the device with builder optimization level 3 [36], except the compression-screening engines of Section 8, built at level 0 (level 3 reference rows are marked in Table A.1). From each graph we build two engines:

- a *float* engine that takes a normalized $1 \times 3 \times S \times S$ FP32 RGB tensor, and
- a *uint8* engine whose graph begins with four ONNX operators (Cast, a BGR→RGB Gather, Transpose, division by 255) and takes the letterboxed $S \times S \times 3$ uint8 BGR image directly. Its ONNX Runtime outputs are identical to the float graph’s (maximum absolute difference 0 on test images).

The Ultralytics baseline uses an engine that Ultralytics exported on the device from the same checkpoint (`half=True`, `nms=False`), with the TensorRT builder defaults (optimization level 3, the same value we pin). It runs through `YOLO.predict` with confidence threshold 0.001 and at most 300 detections, the same settings as our pipelines.

4.4. Pipelines

All our pipelines share one implementation and differ only in the switches below. They define the cumulative ladder of Section 6. Each pipeline goes from a JPEG file to COCO-format rows: OpenCV decode, letterbox to $S \times S$ with value-114 padding, copy into a pinned host buffer, TensorRT execution, output copy, and vectorized conversion of rows with confidence ≥ 0.001 to original-image coordinates.

- *Input type*: float input with host normalization, or uint8 input with in-graph preprocessing.
- *Synchronization*: `cudaStreamSynchronize` on the inference stream, or `cudaEventSynchronize` on an event recorded after the output copy. In Section 6.3 we additionally set the CUDA device scheduling flag (`auto`, `spin` or `blocking-sync`) before context creation.
- *Prefetch*: one or two worker threads decode and letterbox upcoming images while the current image is processed.
- *Overlap*: double-buffered I/O, so that rows of image $i-1$ are built while image i runs on the GPU. The *arrival-aware* variant (Section 7) finishes the in-flight image immediately if the next frame is not yet available.

Table 3

Measurement boundaries used throughout. Terms are distinct quantities: they differ in what host work and queueing they span, and we never equate them.

Term	Span (start → end)
GPU compute (<i>trtexec</i>)	device-side execution of one inference; CUDA events, no host work
Enqueue-to-done ("GPU ms" in tables)	H2D enqueue → completion of its D2H; includes GPU queueing
Accelerator call ("infer", NPU tables)	submit → completion of one NPU call; incl. queue wait in overlaps
Frame latency (p50/p95, "latency")	frame availability → finished COCO rows; backlog: decode start
Completion period	1/throughput over timed window
Energy per image	$\int P dt$ over window ÷ timed images

- *Garbage collection*: by default Python’s cyclic collector runs as usual. One diagnostic variant (Section 6.2) disables it for the timed window.

4.5. Workloads

Backlog: all images are available at once and processed as fast as possible. This gives throughput and energy per image. *Streaming*: frames become available at a fixed rate of 15, 30 or 60 fps, using the first 1,000 val2017 images. Latency is measured from a frame’s availability to its finished detection rows, and includes any queueing when the pipeline cannot keep up. We report the median (denoted p50 throughout) and the p95. We prefer p95 to p99 because with 1,000-frame streams the p99 estimate rests on ten samples; p95 is the highest quantile we consider stable at this size. In both workloads the first 20 images are a warm-up and are excluded. In streaming runs the arrival schedule is re-anchored at the end of the warm-up, so the timed window always starts with an empty queue and contains no warm-up backlog. JPEG files are read once before each phase, so all decodes come from the page cache.

4.6. Measurement boundaries

Several time quantities recur, and they are not interchangeable; Table 3 fixes their boundaries. The model’s *G* (Section 3) is the device-side kernel cost, measured as *GPU compute* in the engine sweep and as *enqueue-to-done* inside our pipelines; on this unified-memory board the two differ only by the launch and copy path, 0.1–0.4 ms at the top clock (4.25 ms device-side versus 4.33–4.65 ms enqueue-to-done for the same locked engine), and at low clocks the launch path itself stretches (Section 6.3). The *M* term of Section 3 is host work outside *G* on the pipeline’s critical path; it never double-counts waiting that is already inside an enqueue-to-done span.

4.7. Power, clock and thermal measurement

A separate process samples VDD_IN voltage and current, the GPU devfreq clock, the highest CPU core clock and the junction temperature (*tj-thermal*) at 20 Hz. Each run records its timed window in wall-clock time. Energy is the trapezoidal integral of power over that window, and energy

per image divides it by the number of timed images. VDD_IN is the module’s main input rail, measured by the module’s INA3221 monitor; carrier-board loads (fan, USB peripherals, display) sit on other rails and are excluded. We measured idle power over 60 s windows twice for each clock policy (Section 6). Unless stated otherwise, the GPU clocks we report are arithmetic means of the samples; for the model of Eq. (1) we use the harmonic mean. To check that the sampler does not perturb what it measures, we ran the sequential and the full pipeline twice each with and without the power sampler process (first 2,000 val2017 images; a lighter logger of GPU load and clocks ran in both cases). Mean throughput differed by less than 0.2% (40.2 versus 40.2 and 209.7 versus 209.4 images/s), well within run-to-run variation.

4.8. Accuracy protocol and data hygiene

Accuracy is official pycocotools COCOeval (bbox, maxDets 100) on all 5,000 COCO val2017 images [1], reported as AP (IoU 0.50:0.95), AP₅₀ (IoU 0.50) where noted, and AP_S (small objects). Every prediction file is hashed. All pipelines that differ only in scheduling produce byte-identical predictions in every repeat and under both clock policies, so for them accuracy is held exactly constant. Every development decision (compression screening, router fitting, router threshold) used two disjoint subsets of COCO train2017: a 1,024-image development split and an independent, hash-verified 1,024-image verification split. Because the pretrained checkpoint was itself trained on train2017, these splits are not unseen by the base model; they are held out only from our own fitting and selection, and their absolute AP is optimistic. We use them only for relative comparisons between variants. val2017 was used only for the measurements in this paper.

4.9. Use of AI assistance in the measurement process

As declared in the back matter, an AI coding agent (Factory Droid) took part in the measurement campaigns. Concretely, the agent executed the campaign scripts on the boards over SSH, monitored runs for completion, transferred logs and traces, and drafted the aggregation and plotting code that turns raw traces into tables and figures. The authors designed every experiment, protocol and comparison, reviewed every script before it ran, and made every analysis decision. No reported number rests on the agent’s own summaries: prediction files are hash-verified across runs, and every table and figure is regenerated from the archived raw traces by the published aggregation scripts (Appendix C).

4.10. Repetitions and statistics

Each backlog configuration on YOLO26n is repeated three times per clock policy. Resolution, generality, streaming and scheduling-flag experiments are repeated twice. Locked-clock generality runs are repeated once. Later sessions added follow-up experiments with the same software and the same protocol: driver-reported GPU load per pipeline (one run each), a lock-then-release test of the governor, a readback verification that *jetson_clocks* holds

the memory clock under load, sampler overhead, a sweep over the number of decode workers, utilization clamping on the sequential pipeline and on the prefetch and full pipelines under blocking synchronization (two runs each), a GIL switch-interval control on the full two-worker pipeline under locked clocks (two runs each), a clocked repeat of the stage breakdown, a minimum-frequency CPU clamp test (two runs), a float-engine zero-gap `trtexec` benchmark (two runs), and a fixed-frequency GPU sweep (three 15 s runs per devfreq step, only the GPU domain pinned, plus two control runs with all clocks unlocked; the memory clock was logged at ≈ 4 Hz in both). In these runs a logger additionally recorded the driver’s GPU load, the GPU clock and the clock of CPUs 0–3 at 20 Hz. The order of configurations is rotated between repeats, with a 10 s pause between runs, and no other GPU work runs during a measurement. We report means and full ranges (minimum to maximum) rather than confidence intervals, because two or three repeats are too few for a meaningful interval. Run-to-run ranges are small (median 0.8% of the mean for backlog throughput). For the held-out router test we use a paired image bootstrap.

5. RQ1: The governor sets the in-application engine time

5.1. Stage attribution in a sequential application

We first timed every stage of a conventional sequential application with CUDA events, using the float-input YOLO26n engine at 640, on 1,004 images of the train2017 verification split. The medians were 7.5 ms for JPEG decode, 3.9 ms for letterbox and normalization, 0.65 ms for the pinned host-to-device copy, 12.4 ms for TensorRT execution and 0.01 ms for the output copy; with row construction and Python-side dispatch (≈ 1.5 ms), the median total is 26.0 ms. This breakdown covers the OpenCV-based row path; the fallback converter used for the COCO evaluation rows is exercised only by the accuracy runs and is not in these numbers. The host-stage times are governor-dependent: this run used the default governors, which held the CPU domain near 955 MHz, so the decode and letterbox figures are those of a down-clocked host. A clocked repeat of the same measurement (domain logged at 20 Hz, 837 MHz mean) reproduced every stage median within 7% (decode 8.0 ms, letterbox 4.2 ms, compute 12.9 ms, total 26.9 ms), so the attribution is stable across the clocks the governor actually visits. Copies are negligible on this unified-memory device, and the NMS-free output needs no suppression. By this account the model takes about half of the application’s time, which would make it the obvious optimization target.

The same engine, however, runs in 4.1 ms back-to-back in `trtexec` (zero gap, two 15 s runs, medians 4.10 and 4.13 ms; the uint8-input engine of Table 4 runs 4.2 ms). On val2017, every sequential pipeline of the ladder in Section 6 records a mean GPU clock of exactly 306 MHz, the governor’s floor. Its median enqueue-to-completion time is 13.2–13.3 ms for both input types, 3.1–3.2 $\times$ the respective back-to-back time.

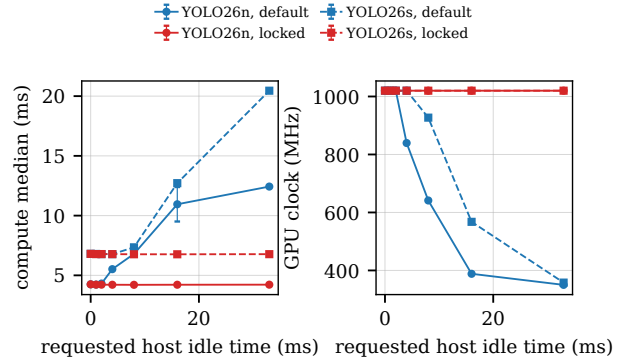


Figure 1: Median GPU compute time of the uint8 FP16 engines in `trtexec` (left) and mean sampled GPU clock (right) versus a forced idle gap between inferences. Mean of two 15 s runs per point. Default governors (blue) versus `jetson_clocks` (red). The `-idleTime` option sets host idle time, not a frame period: a 33 ms setting yields about 26.7 inferences/s.

5.2. An idle-gap sweep isolates the governor

To separate the governor from everything else in the application, we ran the uint8 engines in `trtexec` with a forced idle gap between consecutive inferences, for 15 s per point and twice per point (Fig. 1, Table 4). With locked clocks, YOLO26n’s median compute time is 4.20–4.27 ms at every gap. With default governors it stays at 4.2–4.3 ms up to a 2 ms gap. It then grows to 5.5 ms at 4 ms, 6.8 ms at 8 ms, and 12.4 ms at 33 ms. Note that `-idleTime` inserts host idle time, not a frame period: at a 33 ms setting the achieved rate is about 26.7 inferences/s, and the runs carry no camera-style arrival process. Moreover, `trtexec` pipelines its enqueues, so the GPU-idle interval the governor actually sees is shorter than the requested host sleep; the last row pair of Table 4 reports the achieved rates, from which the mean GPU-idle interval can be derived (3.0–25.4 ms for YOLO26n against the requested 4–33 ms). YOLO26s keeps the GPU busy longer per inference, so it holds its 6.8 ms up to a 4 ms gap and rises to 20.5 ms at 33 ms. Read along the idle-gap axis, a host that leaves ≈ 30 ms between inferences therefore pays about 3 $\times$ the minimum compute time per frame under the default governor; actual camera-rate arrival is measured directly in Section 7.

5.3. Validating the duty-cycle model

Engine time. Table 4 compares the measured compute time with the prediction of Eq. (1). The prediction uses $G_{\max}$ from the locked runs (4.22 ms for YOLO26n, 6.78 ms for YOLO26s) and the harmonic-mean clock sampled during each run. Over the 14 down-clocked sweep runs, the mean absolute error is 4.8%, and the per-cell errors are signed in both directions (−6.7% for YOLO26n at the 4 ms gap to +11.0% for YOLO26n at the 16 ms gap). The largest error (23%) occurs in one of the two 16 ms-gap YOLO26n runs. Its median compute time differs from its repeat’s by 30%, although their harmonic-mean clocks differ by only 8%. This suggests clock changes faster than our 20 Hz sampling

Table 4

Idle-gap sweep (trtexec, uint8 FP16 engines, mean of two 15 s runs). Default governors: measured median compute time, harmonic-mean GPU clock $\bar{f}$, and the prediction $G_{\max}f_{\max}/\bar{f}$ of Eq. (1). “Lock” is the measured median with jetson_clocks. The gap axis is the requested host idle time (`-idleTime`), not a frame period. Because trtexec pipelines its enqueues, the achieved rate (last row pair) implies a mean GPU-idle interval shorter than the requested sleep: 3.0, 5.7, 9.4 and 25.4 ms for YOLO26n where the host requests 4, 8, 16 and 33 ms (1.5, 4.9, 8.1 and 18.0 ms for YOLO26s).

Gap (ms)	YOLO26n				YOLO26s			
	ms	$\bar{f}$	pred.	lock	ms	$\bar{f}$	pred.	lock
0	4.25	1020	4.22	4.25	6.80	1020	6.78	6.80
1	4.23	1020	4.22	4.22	6.79	1020	6.78	6.78
2	4.31	1020	4.22	4.22	6.78	1020	6.78	6.78
4	5.52	836	5.15	4.21	6.77	1020	6.78	6.78
8	6.80	632	6.82	4.21	7.35	926	7.46	6.76
16	10.95	355	12.15	4.21	12.72	550	12.57	6.76
33	12.43	323	13.33	4.22	20.45	327	21.12	6.77

achieved inf/s, YOLO26n: 235, 236, 169, 118, 81, 49, 27

achieved inf/s, YOLO26s: 147, 147, 147, 121, 82, 50, 27

resolves — independent measurements put the GPU clock-actuation lag at about 5 ms on this SoC family [21]. For the non-overlapped application rungs of Table 6, Eq. (1) overestimates the measured enqueue-to-completion time by 0.4–11.5%. The positive bias is expected in part because Eq. (1) attributes all of the engine time to the GPU clock, leaving no room for a clock-insensitive component (memory-bound kernels, launch overhead) that a two-term fit $G(f) = a + b/f$ would absorb. But the approximation itself also contributes: the window’s harmonic-mean clock includes host-only intervals, during which the clock is lower. Reweighting the clock by the driver’s per-sample GPU load in the separate load-logging session reduces the prefetch rungs’ overestimate from +19–36% to +9–13%, so a large share of the apparent bias where the clock moves is a sampling artefact of the window average, not physics. The sequential rungs, pinned at 306 MHz for the whole run, still show a 5–7% overestimate, which is the cleaner estimate of the clock-insensitive share. We therefore treat Eq. (1) as a first order description with a ~5% clock-insensitive residual, not as an exact law.

A fixed-frequency sweep makes the same point without any window-averaging approximation: with the GPU’s devfreq domain pinned at each of its eight steps (and the CPU and memory clocks left at their shipped settings), the same uint8 engine at zero gap takes 12.49, 9.54, 7.86, 6.48, 5.68, 5.05, 4.59 and 4.25 ms at 306, 408, 510, 612, 714, 816, 918 and 1020 MHz (Table 5). A pure inverse fit $G = b/f$ with b fitted by least squares reproduces these medians with a mean absolute error of 3.7% and a systematic residual (+3.1% at the floor, –9.1% at the top clock); the anchored form of Eq. (1) ($b = G_{\max}f_{\max}$) is less accurate (6.6% mean absolute error, always overestimating below the top clock),

Table 5

Fixed-frequency sweep: the uint8 YOLO26n engine at zero idle gap with the GPU pinned at each devfreq step (three 15 s trtexec runs per step; repeat medians agree to 1.1%). The memory clock is logged (constant 2133 MHz at 306–510 MHz, bistable 2133–3199 MHz from 612 MHz up; see text). $G(f) = b/f$ uses the least-squares $b = 3.94$ GHz·ms fitted to the eight points (anchoring $b = G_{\max}f_{\max}$ at the top point instead, as Eq. (1) does, raises the mean absolute error to 6.6%); the two-term fit adds a clock-insensitive a . Errors are signed, (predicted – measured)/measured.

f (MHz)	measured (ms)	b/f (%)	$a+b/f$ (%)
306	12.49	+3.1	+0.1
408	9.54	+1.2	–0.1
510	7.86	–1.7	–1.3
612	6.48	–0.6	+1.4
714	5.68	–2.8	+0.7
816	5.05	–4.5	+0.6
918	4.59	–6.5	0.0
1020	4.25	–9.1	–1.4

and the two-term fit $G(f) = a + b/f$ removes the structure ($a = 0.63$ ms, $b = 3.63$ GHz·ms, mean absolute error 0.7%, all residuals within $\pm 1.5\%$). The pinned 306 MHz time of 12.49 ms also closes the loop with the application: adding the ≈ 0.7 ms launch path at that clock reproduces the sequential pipeline’s 13.2 ms enqueue-to-completion.

Because the memory (EMC) clock also scales with load on this SoC, it is a possible confound for the sweep; prior work reports memory-clock effects of 11–48% on edge-inference latency [21]. We reran the sweep with three repeats per step and the EMC clock logged at ≈ 4 Hz, plus two control runs with the EMC governor fully unlocked. An attempted EMC lock did not hold; independent measurements on the same SoC family document that EMC clock control fails silently on these board support packages unless a second lock bit is asserted [21], so we control the confound by measurement rather than by pinning. The logged traces show the memory clock bistable: constant 2133 MHz at GPU clocks of 306–510 MHz, and alternating 2133–3199 MHz from 612 MHz up. Two checks bound its influence: fitting $a+b/f$ to only the EMC-constant points changes b by 2% (3.55 versus 3.63 GHz·ms), and the unlocked control runs reproduce the pinned medians within 0.4%. These controls show that the inverse trend is primarily a GPU-clock effect and cannot be explained by memory-clock scaling alone.

Governor operating points. The busy fraction u of Section 3 can be estimated from timing, but the GPU driver also reports its own load estimate, in per mille, through a sysfs node. In a separate session we reran each of our eight pipeline rungs on the first 2,000 val2017 images and logged this load together with the GPU and CPU clocks at 20 Hz. The measured load agrees with the timing estimate to within 0.05 for every rung (the estimate is 0.01–0.05 higher), so we use the measured value for the application rungs below.

Fig. 2 plots every operating point as GPU load against harmonic-mean clock. Whenever the governor settles clearly inside its range (400–1000 MHz), the load lies between 0.53 and 0.74: 0.53–0.64 for the `trtexec` sweep (timing estimate) and 0.64–0.74 for the application rungs (measured). Two caveats bound this band: the sweep and the application runs use different estimators, with the timing estimate biased 0.01–0.05 high, so the lower edge cannot be separated from that bias; and the 0.74 upper edge comes from a rung settled at 993 MHz, close to the 1020 MHz clamp, so the true upper threshold may lie higher. With those caveats, this is the load-band behavior assumed in Section 3. The sequential rungs sit at the floor with measured loads of 0.46–0.66. Even at the floor, their load sits at or below the observed band’s top, which is consistent with the governor never raising the clock. This is the fixed point described in Section 3. Removing host stages from the GPU’s critical path pushes the load to the top of the band or above it, and the clock follows: prefetch with event synchronization settles at a harmonic-mean clock of 853 MHz in the load-logging session and 881–891 MHz across the three backlog repeats (the stream-synchronized variant of the same rung stays at 655 MHz in this session and at 631–694 MHz in the blocking-sync runs of Section 6.3), and the full schedule, at a measured load of 0.87 (timing estimate 0.90), stays at 1020 MHz.

The floor is the only stable point we observe. A load-band controller could, in principle, hold a sequential pipeline at a high clock if it started there. We tested this by starting the sequential stream-synchronized pipeline with locked clocks and restoring the default governor after 10 s, twice. At 1020 MHz the pipeline’s load was only 0.35–0.36, below the band. Within 0.25 s of the release the governor began to step down (1020, 714, 612, 510, 408 MHz), held 408 MHz for about 2.5 s at a load of about 0.5, and reached 306 MHz 3.3–3.4 s after the release. It stayed there for the rest of the run at a load of 0.52–0.53. We find no second operating point in any run. The load-band model does not exclude interior equilibria in principle — a pipeline whose load stops responding to the clock could settle elsewhere — but for this workload the release lands at the floor from every starting clock we tried.

Clock and power traces. Fig. 3 shows the first 12 s of four application runs. Both sequential variants stay at 306 MHz for the whole run. Prefetch climbs in steps from the floor and then moves between about 700 and 1020 MHz. The full schedule is pinned at 1020 MHz. Input power follows the clock. Both optimized schedules also show short power dips whose spacing grows over the run — from about 1.9 to 4.3 s between successive dips for the full schedule, with longer, less regular intervals (up to about 7 s) for prefetch. We traced these dips to Python’s cyclic garbage collector, whose pauses grow as the list of detection rows grows. We quantify their effect over whole runs in Section 6.

Answer to RQ1. Under the default governor, the in-application cost of a fixed engine is set by the application’s GPU duty cycle. Engine time follows the inverse of the clock

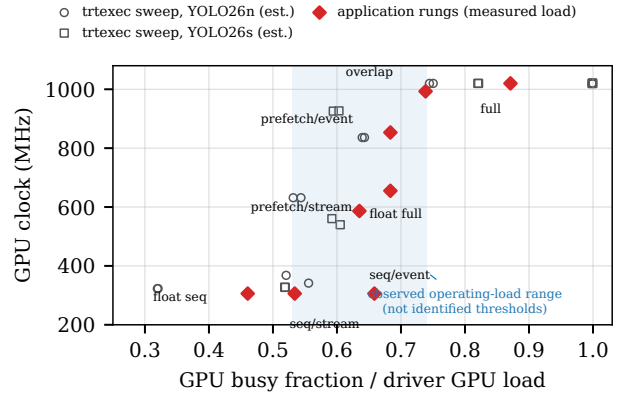


Figure 2: Governor operating points: GPU busy fraction or measured load versus harmonic-mean GPU clock. Open markers: `trtexec` idle-gap sweep, busy fraction estimated from timing. Diamonds: default-governor application rungs (first 2,000 val2017 images), load measured by the GPU driver. The shaded band (0.53–0.74) is the observed operating-load range — it contains every operating point with a clock between 400 and 1000 MHz; it is not an identified pair of controller thresholds.

(Eq. (1)). Every operating point we observe inside the governor’s range sits in a GPU-load band of 0.53–0.74 — 0.64–0.74 measured by the driver’s load counter on the application runs, 0.53–0.64 from a timing estimate on the engine sweep, biased 0.01–0.05 high — a band consistent with the load-band behavior of Section 3, whose exact internal thresholds we do not identify. The only other state we observe is the floor: a sequential pipeline reaches it from any starting clock and stays there, which makes its engine about 3× slower than the benchmark. This effect is larger than any engine-time change produced by the compression methods we tried (Section 8).

6. RQ2: Which pipeline decisions matter

6.1. Cumulative ablation on COCO val2017

Fig. 4 illustrates the three schedules. Table 6 and Fig. 5 report the cumulative ladder on all 5,000 val2017 images. Model, engine and accuracy are fixed. All uint8 rows share one prediction file, and both float rows share another.

Accuracy. Our uint8 pipeline scores 0.40257 AP and 0.19766 AP_S, against 0.40259 and 0.19779 for the Ultralytics predictor on its own engine. The float-input variant scores 0.40240. For reference, the YOLO26 paper reports 40.9 AP for YOLO26n, measured on a desktop GPU with its own export [7]; on our device and evaluation settings the Ultralytics predictor itself scores 0.40259, so the ≈0.6-point difference from the paper is a property of the evaluation stack, not of our pipeline. Reaching this agreement required one fix to our decoder. An earlier version subtracted the fractional letterbox padding, rather than the integer offset actually used to place the resized image. This shifted boxes by half a pixel whenever the padding was odd and measurably lowered AP on the verification split. Such errors are easy to make and

Table 6

Cumulative pipeline ablation for YOLO26n at 640 on all 5,000 COCO val2017 images (batch 1, FP16 TensorRT, MAXN_SUPER). Each cell is the mean of three rotated runs; run-to-run throughput ranges are below 2.5% of the mean for every cell except the float sequential pipeline under default governors (33.1–34.3 images/s) and prefetch with stream synchronization under default governors (111.5–119.1 images/s). The † row is a diagnostic, not a cumulative step: it reverts the previous rung’s event synchronization to isolate its effect (Section 6.3). The last two rows are variants of the full uint8 pipeline. AP is official COCOeval. Each row’s predictions were byte-identical across all six runs (both clock settings), and rows with the same input type share one prediction file, except the Ultralytics baseline, which exports its own engine and has its own prediction hash. GPU MHz is the mean sampled devfreq clock under the default governor (always 1020 when locked). CPU ms is process CPU time per image; W is mean module input power (VDD_IN rail). Table headers abbreviate images per second as img/s.

Pipeline	AP	AP _s	Default governors					jetson_clocks		
			img/s	mJ/img	W	GPU MHz	CPU ms	img/s	mJ/img	W
Ultralytics predict	0.4026	0.1978	31.1	194.6	6.06	306	20.5	73.9	139.3	10.29
Ours: float input, sequential	0.4024	0.1975	33.9	187.6	6.36	306	21.9	59.9	165.3	9.90
+ in-graph uint8 preprocessing	0.4026	0.1977	40.1	156.1	6.26	306	17.8	90.1	119.1	10.74
+ event synchronization	0.4026	0.1977	50.0	150.7	7.54	306	21.5	90.4	119.9	10.84
† + decode prefetch (stream sync)	0.4026	0.1977	116.1	83.7	9.72	736	13.5	158.6	86.2	13.67
+ decode prefetch	0.4026	0.1977	146.6	87.2	12.77	911	12.7	159.4	87.2	13.90
+ post-processing overlap	0.4026	0.1977	175.2	81.2	14.23	999	10.6	182.6	80.5	14.69
+ second decode worker	0.4026	0.1977	213.8	74.5	15.94	1019	11.6	217.9	74.5	16.24
Full scheduling, float input	0.4024	0.1975	89.2	109.9	9.80	599	21.6	98.5	122.2	12.03
Full, arrival-aware overlap	0.4026	0.1977	194.8	77.9	15.17	1011	12.8	214.2	75.2	16.12
Full, GC paused in timed loop	0.4026	0.1977	230.2	71.6	16.49	1016	11.2	231.8	72.4	16.79

hard to notice in a custom pipeline, so we checked every pipeline in this paper with official COCOeval on the full split.

Baselines. Under default governors the Ultralytics predictor processes 31.1 images/s at 194.6 mJ per image. Our float-input sequential pipeline is similar (33.9 images/s). Both run the GPU at 306 MHz for the whole run. Locking clocks raises them to 73.9 and 59.9 images/s, at 3.5–4.2 W more module input power. Locking clocks with `jetson_clocks` is the usual recommendation for benchmarking on Jetson [33].

Host-side changes help a sequential schedule only through the host. Moving normalization and the channel transpose into the engine saves 4.1 ms of CPU time per image and raises throughput by 18% (33.9 to 40.1 images/s). Waiting on an event instead of the stream raises it to 50.0 images/s; put the other way, the stream-synchronized pipeline loses 20% of throughput. The GPU still runs at 306 MHz and the engine still takes 13.2 ms. The event-sync gain comes from the host side, at a cost of 1.3 W and 3.7 ms of CPU time per image (Section 6.3).

Overlapping CPU and GPU work changes the governor’s regime. Prefetching the next image’s decode and letterbox gives the largest single step, from 50.0 to 146.6 images/s. The GPU is now idle only briefly between inferences, and the governor raises its mean clock to 911 MHz. The engine’s enqueue-to-completion time falls from 13.2 to 4.4 ms, close to its back-to-back value. Overlapping row construction of image $i-1$ with inference of image i gives 175.2 images/s at 999 MHz. A second decode worker gives 213.8 images/s at 1019 MHz. More workers do not help.

In a separate sweep over one to five workers (two runs each), throughput was 174.9, 214.2, 215.9, 216.4 and 216.6 images/s, so the third to fifth workers add 1.1% together. With two workers the GPU’s measured load is already 0.90 (worker-sweep session; 0.87 in the load-logging session of Section 5) and its clock is at the maximum, so the pipeline is limited by the engine and the main thread, not by decode. Extra workers only lengthen the queue: median latency per image (from the image entering the pipeline to its finished rows) grows from 17.2 ms with two workers to 30.1 ms with five. The full pipeline is 6.9× faster than the Ultralytics predictor under the same default governors. The Ultralytics predictor is a weak baseline, so we also compare against the strongest sequential configuration we measured, uint8 input with event synchronization and all clocks locked (90.4 images/s). The full pipeline under default governors is still 2.4× faster, at 0.4026 AP in both cases.

Locking clocks becomes unnecessary. With locked clocks the full pipeline reaches 217.9 images/s, only 1.9% more than under the default governor, at the same 74.5 mJ per image and 0.3 W more power. Once the pipeline keeps the GPU busy, the stock governor already selects the top clock. Locking helps the sequential pipelines by 1.8–2.3× (2.4× for the Ultralytics predictor) and the full pipeline hardly at all.

Preprocessing placement interacts with the schedule. The float-input variant of the full pipeline in Table 6 reaches only 89.2 images/s. Host-side normalization raises process CPU time from 11.6 to 21.6 ms per image, the CPU becomes the bottleneck (Eq. (4)), and the GPU clock averages only

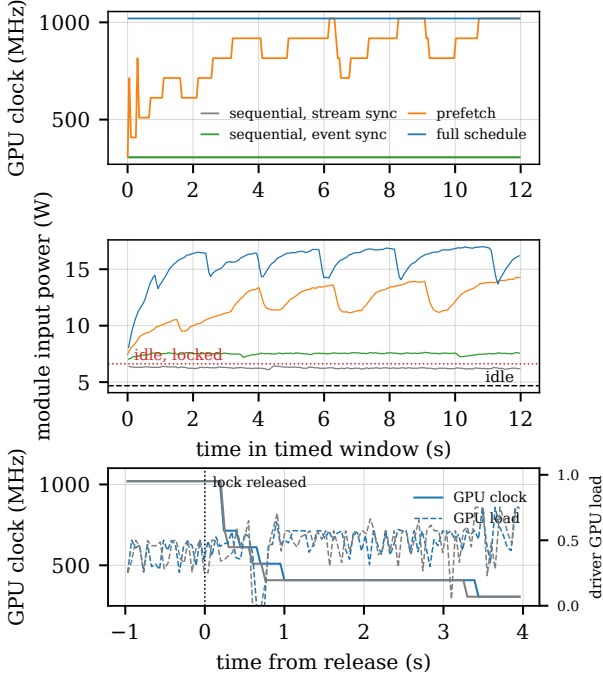


Figure 3: GPU clock (top) and module input power (middle) during the first 12s of four YOLO26n application runs on val2017 under default governors (20 Hz samples); the two sequential traces coincide at 306 MHz in the top panel. Bottom: the lock-then-release experiment — the sequential pipeline is started with the GPU clock locked at 1020 MHz and the lock is released at $t=0$ (dotted line); clock (solid) and driver GPU load (dashed, right axis) of two repeats fall to the 306 MHz floor within about 3.4 s, and no second stable operating point appears.

599 MHz. In-graph preprocessing is thus worth 18% in a sequential pipeline but $2.4\times$ in the full one. Ultralytics 8.4.172 already copies uint8 data and normalizes on the GPU, so the technique is not new. Our point is that its value depends on whether the schedule can keep the GPU busy.

Where the energy saving comes from. Energy per image falls from 194.6 mJ (Ultralytics) to 74.5 mJ, a factor of 2.6. Eq. (5) shows that most of this saving comes from amortizing the idle floor (4.68 W) over more images per second, not from doing less work per image. The marginal energy above idle is 44.1 mJ per image for Ultralytics and 52.6 mJ for the full pipeline. Across all default-governor runs it ranges from 39 to 57 mJ, while the idle-floor term ranges from 22 to 150 mJ. The full pipeline does slightly more work per image above idle (higher clocks, two decoders), but it finishes much sooner. Its energy advantage is real when the device works through a backlog or can sleep after finishing. At a fixed camera rate the situation is different (Section 7).

6.2. Garbage-collection pauses in the evaluation loop

The power traces of the optimized schedules contain short dips (Fig. 3). In the timed window of the full pipeline

under the default governor, we count six to seven dips of more than 2 W below the median power per run, with onsets at about 4.1, 6.0, 8.3, 11.3, 14.8 and 19.1 s. The intervals between dips grow, from 1.9 to 4.3 s. This pattern matches CPython’s generational garbage collector. A full collection runs only when the number of long-lived objects has grown by 25% since the last one, and every full collection traverses all of them [37]. Our evaluation harness, like most COCO evaluation scripts, keeps every detection as a Python dictionary until the end of the run. For val2017 these are 732,651 detections, each a dictionary holding a box list, or about 1.5 million objects tracked by the collector. Each full collection therefore stalls the main thread for longer, the GPU drains, and the power drops.

To confirm the cause, we added a variant that performs one collection before the timed window and then disables the cyclic collector until the window ends. Reference counting still frees all acyclic objects, so memory use is unaffected for this workload. The predictions are byte-identical to the other uint8 rows (Table 6). The dips disappear: no run contains one, and the standard deviation of the sampled input power falls from 0.88–0.90 W to 0.16–0.20 W under the default governor and from 0.79–0.80 W to 0.07–0.09 W with locked clocks. Throughput rises from 213.8 to 230.2 images/s, so the collector costs 7.1% of the measured throughput, and energy per image falls from 74.5 to 71.6 mJ. With locked clocks the cost is 6.0% (217.9 versus 231.8 images/s).

We do not count this gain as a pipeline improvement. A deployed detector would pass detections on rather than accumulate them, and its collector would have little to scan. The effect matters for benchmarking. An evaluation loop that accumulates results penalizes a fast pipeline more than a slow one, because the same pauses are a larger share of a shorter run. The pauses also arrive at irregular, growing intervals, which inflates tail latency in long runs. Whenever we compare against the Ultralytics predictor we therefore use the rows with the collector enabled, which is the more conservative choice for our pipeline. The rest of the paper follows the same rule.

6.3. Synchronization: spinning versus sleeping

The 20% effect of the synchronization call is surprising, so we isolated it. By default, `cudaEventSynchronize` on an event created without the blocking-sync flag busy-waits, whereas `cudaStreamSynchronize` follows the device scheduling policy, which on this system lets the host thread yield or sleep. We crossed the two calls with the three device scheduling flags [38] on the first 2,000 val2017 images (Table 7).

The effect is spinning versus sleeping. In the sequential schedule, event synchronization gives 49.9–50.0 images/s under all three flags. Stream synchronization gives 40.5 images/s with auto and 40.0 with blocking-sync, but 50.0 with spin. The effect — 19.8% on the ladder rungs of Table 6 and 16.7% in the pinned-CPU experiment below — is therefore not a property of events versus streams. It depends only on

The governor is part of the model

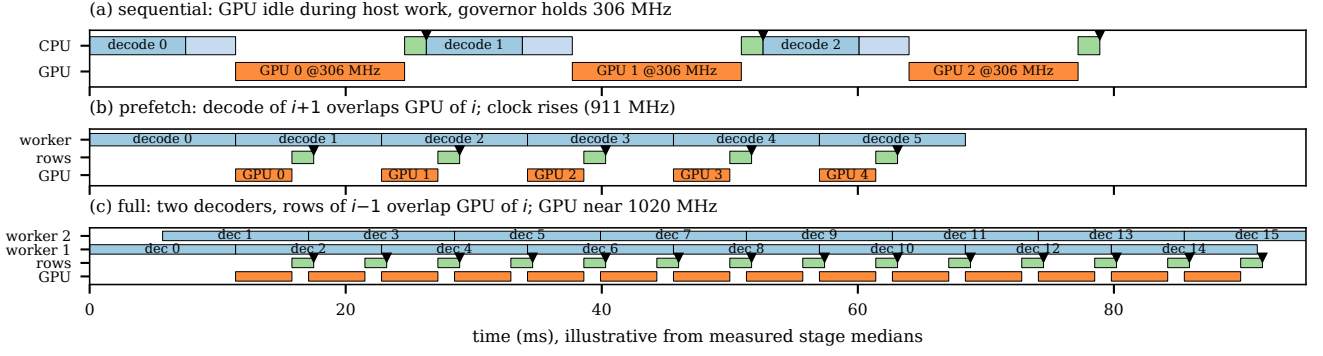


Figure 4: Schematic of the three schedules. The layout is schematic, but the time axis uses the measured stage medians of the sequential float pipeline of Section 5 (decode 7.5 ms, letterbox 3.9 ms, rows ≈ 1.5 ms, engine 13.2 ms host-measured enqueue-to-completion at 306 MHz; the device-side CUDA-event share is 12.5 ms, Table 5; the stage medians sum to 26.0 ms, while the measured period is 29.5 ms — 33.9 images/s — the difference being per-image main-thread work outside the timed stages). Markers ($\blacktriangledown$) show when each frame’s detections become ready. The bars illustrate the structure, not the measured throughputs. In (a) every host stage is GPU idle time, so the governor holds the minimum clock. In (b) and (c) the host stages overlap engine execution.

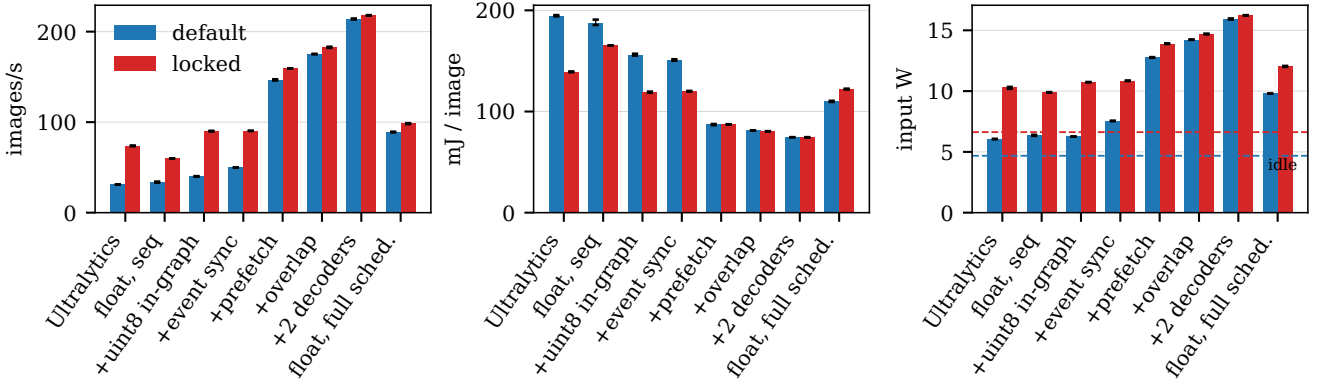


Figure 5: Throughput, energy per image and mean input power along the ladder of Table 6 (bars: mean; whiskers: minimum–maximum of three runs). Dashed lines in the right panel mark idle input power (4.68 W default, 6.62 W locked).

whether the waiting host thread spins or sleeps. On our software stack, `cudaEventSynchronize` spun under every device flag, which is consistent with the CUDA documentation tying blocking waits on an event to the `cudaEventBlockingSync` creation flag [38]. Our events do not set it. `auto` behaved like `blocking-sync` for stream synchronization. Spinning costs 3.7–4.2 ms of process CPU time per image and 1.2 W of module input power, but because each image finishes sooner, energy per image is 4% lower (151.1 versus 157.2 mJ).

Engine time is unchanged; host work slows down.

The GPU clock is 306 MHz and the engine’s enqueue-to-completion time is 13.2–13.3 ms in all six sequential cells. The waking thread therefore does not miss the completion by a measurable amount. (The span is not pure GPU time: with the host clamped at 115 MHz it grows to 31.5 ms, because the enqueue-to-completion window contains the host’s launch path once the host is an order of magnitude slower than the engine.) The difference lies in the host stages. The per-image period grows from 20.0 to 24.7–25.0 ms, and the sampled clock of the fastest CPU core falls from 1728 MHz to about 1643–1694 MHz on average. We hypothesized that

the CPU governor (`schedutil`) causes the slowdown. It sets the clock of each frequency domain in proportion to the scheduler’s utilization estimate of its CPUs [39]. While the host thread sleeps through the 13 ms engine phase, its core appears idle, and the governor lowers the clock of the domain. The decode, letterbox and row stages of the next image then run at the lower clock.

A controlled test. To test this without changing the CPU governor, we pinned the sequential pipeline to CPU 0 and logged the clock of its frequency domain (CPUs 0–3) at 20 Hz (Table 8). With stream synchronization the domain averaged 1069 MHz, 62% of its maximum, and throughput was 41.0 images/s. We then started a separate busy-loop process on CPU 1. It does no work for the pipeline, but it keeps the shared frequency domain at 1728 MHz. Throughput rose to 49.0 images/s, which closes 97% of the gap to event synchronization (49.2 images/s), and the pipeline’s own CPU time fell from 15.1 to 10.1 ms per image. The busy loop had no effect on the event-synchronized pipeline, which already held the clock at its maximum. The effect is thus an interaction between the CUDA wait policy and the CPU

Table 7

Synchronization primitive $\times$ CUDA device scheduling flag (first 2,000 val2017 images, default governors, mean of two runs, uint8 engine). “Enq. $\rightarrow$ done” is the median enqueue-to-completion wall time of one inference (Table 3); CPU MHz is the mean of the fastest core’s sampled clock — not necessarily the core that runs the feeder thread. Run-to-run throughput spread exceeds 3% only in the two sleeping-wait stream cells (8.6% and 3.5%, clock wandering between repeats); all other cells are within 3%, and energy-per-image spread is below 1.3% throughout.

Mode	Sync	Flag	img/s	mJ	W	GPU MHz	CPU MHz	CPU ms	Enq. $\rightarrow$ done ms
sequential	stream	auto	40.5	157.2	6.37	306	1643	17.3	13.27
		spin	50.0	151.1	7.55	306	1728	21.5	13.20
		blocking	40.0	156.2	6.26	306	1694	17.8	13.30
	event	auto	50.0	150.6	7.52	306	1728	21.4	13.20
		spin	50.0	151.3	7.56	306	1728	21.4	13.21
		blocking	49.9	151.0	7.53	306	1728	21.4	13.24
prefetch	stream	auto	107.4	83.8	9.01	674	1647	14.2	6.25
		spin	140.4	87.2	12.24	860	1727	13.1	4.63
		blocking	110.6	84.1	9.31	694	1642	13.8	6.11
	event	auto	141.5	86.8	12.28	866	1728	13.0	4.62
		spin	140.5	86.7	12.18	862	1728	13.1	4.62
		blocking	140.5	86.8	12.20	862	1728	13.1	4.63

Table 8

CPU-clock test. Sequential uint8 pipeline pinned to CPU 0 (first 2,000 val2017 images, default governors, mean of two runs). “Busy loop” marks a separate process spinning on CPU 1, which shares CPU 0’s frequency domain; “uclamp” sets the pipeline’s `uclamp.min` to 1024 (bottom block, a separate session with its own stream-sync and event-sync references). MHz is the sampled clock of that domain’s cluster; CPU ms counts only the pipeline process. All runs produced identical predictions.

Sync	Remedy	img/s	p50 ms	CPU ms	MHz	GPU ms	W mJ/img
stream	—	41.0	23.8	15.1	1069	13.29	6.30
stream	busy loop	49.0	19.9	10.1	1728	13.24	7.69
event	—	49.2	19.8	20.2	1728	13.21	7.47
event	busy loop	49.2	19.8	20.2	1728	13.19	7.96
stream	—	41.3	23.6	14.7	1088	13.29	6.33
stream	uclamp	48.0	20.3	10.5	1450	13.23	6.81
event	—	49.0	19.9	20.2	1728	13.23	7.47

frequency governor, with one residual confound: a busy loop also keeps the cluster out of deep idle states, which shortens wake-up latency. We bounded that confound directly by clamping the whole domain at its 115 MHz minimum, so the governor cannot respond no matter how busy the loop keeps the cluster: the loop then changes nothing (7.6 versus 7.7 images/s, under 1%, mean of two runs). Its +20% at the shipped governor is therefore a clock effect. This clamp test cannot rule out sub-millisecond idle-state or wakeup effects at operating clocks — at 115 MHz a fixed microsecond-scale cost is invisible next to 130 ms frames — but the `uclamp` experiment below, which acts on frequency alone and recovers about 87% of the loss, bounds the idle-state share to at most the remaining 13%. A spinning wait keeps the CPU clock high as a side effect.

An unprivileged remedy. A busy loop on another core is a diagnostic, not a fix. Linux offers a direct way to

tell `schedutil` that a task needs a high clock: utilization clamping [9]. A task can raise its own `uclamp.min` with `sched_setattr`, without root privileges on our system, and threads it creates inherit the clamp. We set `uclamp.min` to its maximum (1024) for the stream-synchronized pipeline, with no other change (Table 8, bottom). The domain clock rose from 1088 to 1450 MHz, CPU time per image fell from 14.7 to 10.5 ms, and throughput rose from 41.3 to 48.0 images/s. A same-session event-synchronized reference reaches 49.0 images/s (Table 8), so the clamp closes about 87% of the sleeping-wait penalty. The recovery is partial because the clamp acts only while the task is runnable, so the clock still decays while the thread sleeps through the engine phase. Module input power rose by only 0.48 W, so energy per image fell by 7%, from 153.2 to 142.0 mJ. This is lower than with the spinning wait (151.1–151.3 mJ) or the busy loop (157 mJ), because the clamp raises the clock without burning a core. Per-task clamping is therefore a practical alternative to spinning when a sleeping wait is required, for example to free the core for other work.

The two governors compound. With prefetch, sleeping waits reduce throughput from about 141 to 107–111 images/s, a 21–24% loss. Here the GPU also suffers. The slower host leaves the GPU idle for longer, the GPU governor settles at 674–694 instead of 860–866 MHz, and the engine time rises from 4.6 to 6.1–6.3 ms. A drop in the CPU clock thus lowers the GPU clock as well. The underlying antagonism — while the GPU computes, the host looks idle and is down-clocked, so it runs slowly exactly when the GPU next needs data — has been described for the stock governors of the same board family [17]. Our measurements add to that description in four ways: we trace the trigger to the CUDA wait policy (spinning versus sleeping, not GPU work per se), we bound the effect with the busy-loop and minimum-clock controls above, we test a per-task remedy that needs no root

privileges, and we observe the effect propagating onward to the GPU clock through the duty cycle.

The clamp also lifts the GPU clock. If the compounding runs from the host clock to the GPU clock, re-arming the CPU governor should restore both. We set `uclamp.min` to 1024 on the prefetch rung, where the sleeping wait costs the most (Table 9, two repeats per cell, identical predictions throughout). In the prefetch schedule with blocking synchronization the clamp raises the feeder cores’ clock (1615–1620 to 1714–1716 MHz), the GPU’s duty cycle rises with the faster submission, and the GPU governor settles at 827–860 instead of 631–657 MHz — the range of the spinning and event-synchronized variants. Engine time falls from 6.3–6.9 to 4.7 ms and throughput rises 34% (101.8–104.2 to 136.3–140.2 images/s) at unchanged energy per image (84.3 to 85.2–86.7 mJ): the clamp recovers essentially all of prefetch’s sleeping-wait penalty, without burning a core.

On the full pipeline the wait policy is free. The full two-worker schedule tells a simpler story (Table 9, bottom). With the decode pool active, a *sleeping* event wait — per-frame events created with `cudaEventBlockingSync` — reaches 215.5–215.9 images/s, indistinguishable from the spinning reference (215.3–215.6), with 0.3 ms less process CPU time per image and 0.4–0.5 mJ less energy per image. Locking the clocks changes neither arm (215.1–215.6 sleeping, 214.7–215.5 spinning), and the clamp has nothing left to recover (215.0–215.5). On this schedule the host never goes idle on the critical path — while the main thread sleeps through one completion, two decode workers keep the domain busy — so there is no utilization dip for the governor to misread. The wait policy matters only where the pipeline actually waits: the sequential rung, where each frame sleeps through the engine phase, and prefetch, where a single feeder does the same.

This paragraph replaces an earlier, incorrect version of the same experiment, which we report for transparency. That version passed `-overlap -workers 2 without -prefetch`; the harness creates its decode pool only under `-prefetch`, so those runs decoded on the main thread and their throughputs (40.0–40.7 images/s under default governors, 86.3–89.3 locked) match the sequential rung (40.1; 90.1 locked, Table 6) within 2%. What we had reported as a governor-independent 19% wait penalty of the two-worker schedule was the sequential schedule’s governor-mediated penalty, measured in disguise; a GIL switch-interval control run on the same misconfigured cells was equally vacuous. The harness now rejects `-workers without -prefetch`, and the corrected GIL control on the real two-worker schedule (shortening the switch interval tenfold, 5 to 0.5 ms, under locked clocks) changes nothing either: 215.0–215.8 versus 215.1–215.6 images/s. The corrected runs also confirm the per-frame completion boundary as the quantity that matters, next.

Stream synchronization costs a quarter — through its boundary, not its wait. With the decode pool active, blocking *stream* synchronization in the enqueue-ahead loop still loses 24% (163.9–165.1 versus 215.3–215.9 images/s),

Table 9

Wait policy and utilization clamping on the optimized pipelines (uint8 engine, first 2,000 val2017 images, mean of two repeats; identical predictions throughout, sha256 prefix `df5a48fe35cd`). Prefetch rows: blocking stream synchronization, default governors, a separate earlier session from Table 8 with its own same-session references. Full-pipeline rows (this session): every arm runs the decode pool (`-prefetch -overlap -workers 2`); “event, sleeps” uses per-frame events created with `cudaEventBlockingSync`; locked is `jetson_clocks`. CPU ms the process CPU time per image.

Pipeline	wait	clocks	clamp	img/s	CPU ms	mJ/img
prefetch	stream, sleeps	default	–	103.0	14.8	84.3
prefetch	stream, sleeps	default	uclamp	138.3	10.7	86.0
full w2	event, spins	default	–	215.5	10.6	72.9
full w2	event, sleeps	default	–	215.7	10.2	72.4
full w2	event, sleeps	default	uclamp	215.3	10.3	72.7
full w2	event, sleeps	locked	–	215.3	10.3	72.8
full w2	stream, sleeps	default	–	164.8	10.2	83.3
full w2	stream, sleeps	locked	–	164.3	10.2	83.1

and it loses the same 24% with locked clocks (163.9–164.7), where the GPU stays pinned at 1020 MHz. The cost is therefore not the sleeping wait and not the governor: it is the completion boundary. `cudaStreamSynchronize` waits for *all* work in the stream, including the frame just enqueued, so the rows of frame $i-1$ are finished only after frame i has executed — the loop degenerates to in-order completion with one frame of slack, and the measured enqueue-to-completion window stretches from 7.2 to 10.0 ms. The per-frame event, spinning or sleeping, is what preserves the overlap.

Locking clocks removes the governor-mediated penalty — where there is one. `jetson_clocks` fixes the CPU clock at its maximum as well as the GPU clock. On the sequential and prefetch schedules, stream and event synchronization then differ by less than 1% (sequential: 90.1 versus 90.4 images/s; prefetch: 158.6 versus 159.4; Table 6) — the roughly 20% penalty on these schedules is governor-mediated. The full two-worker schedule has no wait penalty to remove: sleeping and spinning event waits agree within 1% under both governor regimes, and only the wrong synchronization boundary (stream sync, above) costs throughput, identically at both clock settings. A benchmark run with locked clocks would therefore see no synchronization effect on any of these schedules except the stream-sync boundary, while under the default governors the sleeping wait alone costs a fifth of the sequential and prefetch schedules’ throughput.

The contrast between schedules is itself governor-dependent. The full two-worker pipeline outperforms the event-synchronized sequential pipeline by 4.28× under the default governors (213.8 versus 50.0 images/s, Table 6) but only 2.41× with locked clocks (217.9 versus 90.4). Locking the clocks does not raise the overlapped pipeline’s ceiling; it removes the sequential baseline’s self-inflicted penalty, so the reported benefit of scheduling depends on the power-management regime of the benchmark.

Table 10

Streaming at fixed arrival rates (first 1,000 val2017 images, YOLO26n 640, default governors, mean of two runs). Latency runs from frame arrival to finished detection rows (ms); energy is module input energy per frame (mJ). “sat.” marks a pipeline that cannot sustain the arrival rate; its queue grows without bound, and the achieved rate is given instead of latency. †Near-saturated cell: the achieved rate is 29.8 images/s against 30 fps arrivals in both repeats, so the queue drains too slowly to clear between bursts and p95 includes the resulting queuing. Idle input power alone amounts to 312, 156 and 78 mJ per frame at 15, 30 and 60 fps.

Pipeline	15 fps			30 fps			60 fps		
	p50	p95	mJ	p50	p95	mJ	p50	p95	mJ
Ultralytics	32.0	38.4	357	36.2	188.0 [†]	201	sat. (31/s)		194
Float, sequential	39.2	45.2	361	32.2	84.3	204	sat. (34/s)		187
uint8, seq., event	27.4	33.4	353	21.6	25.8	203	sat. (50/s)		151
+ prefetch	26.8	33.3	354	26.7	33.2	198	20.9	99.3	123
+ overlap, 2 workers	85.3	91.8	352	51.9	58.7	196	35.5	49.1	117
+ arrival-aware overlap	27.0	33.1	354	26.5	33.2	198	20.7	40.6	122

Answer to RQ2. The decisions that matter are those that change the GPU’s duty cycle: prefetching (2.9×), overlapping post-processing and adding a decode worker (a further 1.5×), and how the host waits for the GPU. The wait penalty is governor-mediated: on the sequential and prefetch schedules a sleeping wait lets the CPU governor lower the host’s clock, and through the slower host also the GPU’s, and it nearly vanishes when clocks are locked. On the full two-worker schedule the wait policy costs nothing at all — a sleeping event wait matches a spinning one within 1% under either governor regime (Section 6.3); what still costs 24% there is the wrong synchronization boundary (stream synchronization in an enqueue-ahead loop), identically at locked and default clocks. In-graph preprocessing matters once the schedule is no longer sequential. Locking clocks substitutes for a good schedule only partially (1.8–2.3× for our sequential pipelines, 2.4× for the Ultralytics predictor) and is nearly worthless once the schedule is good (1.9%). Finally, the evaluation harness itself can bias the comparison: garbage-collection pauses cost the full pipeline 7.1% of its measured throughput.

7. RQ3: Latency and energy at camera rates

A backlog is the right workload for offline processing, but a camera delivers frames at a fixed rate. We released the first 1,000 val2017 images at 15, 30 and 60 fps and measured, for each frame, the time from its release to its finished detection rows, together with module input energy per frame (Table 10 and Fig. 6). The arrival-aware variant (Section 4.4) is included here. On a backlog it reaches 194.8 images/s, 8.9% less than the full pipeline (Table 6).

Capacity. A pipeline that cannot sustain the arrival rate accumulates a queue, and its latency grows with the run length. At 60 fps this happens to the Ultralytics predictor (31.0 images/s achieved), the float sequential pipeline (34.2) and the uint8 sequential pipeline with event synchronization (49.7); Table 10 rounds these to whole images per second. The Ultralytics predictor is also at the edge of its capacity at 30 fps (29.8 images/s processed). Short stalls create

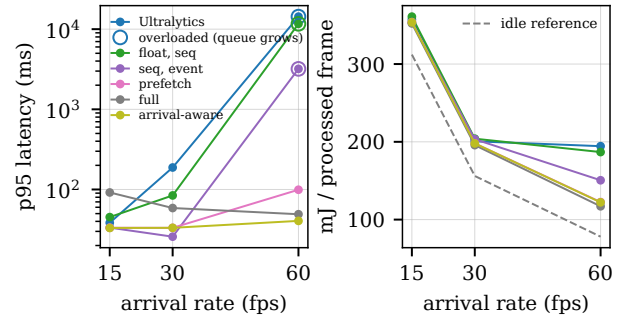


Figure 6: Streaming at 15, 30 and 60 fps under default governors. Left: p95 latency from frame release to finished rows (log scale); open markers mark overloaded cells, where the processed rate falls short of arrivals and the latency reflects a queue that grows over the finite run, not a steady state. Right: module input energy per *processed* frame (the denominator when a pipeline is overloaded); the dashed line is idle power divided by the frame rate, the idle-reference contribution under the measured power state.

queues that drain only slowly, and its latency distribution is strongly right-skewed (mean 55–64 ms against medians of 36–37 ms). An earlier harness version anchored the arrival clock before its own warm-up, which inflated exactly this tail (Appendix B). Dropping the first 100 frames of every run moves the below-capacity p95 values by at most 4 ms (188 to 192 ms for the Ultralytics predictor at 30 fps), so the remaining tails are steady-state queue drain, not start-up transients. Capacity on a backlog therefore matters even for a camera application. It is the margin that absorbs transient stalls.

Energy is set by the idle floor. At 15 fps every pipeline uses between 352 and 361 mJ per frame. The idle floor alone is 312 mJ, or 86–89% of the total. At 30 fps the spread is 196–204 mJ against a floor of 156 mJ. The full pipeline, which uses 2.6× less energy per image than the Ultralytics predictor on a backlog, uses only 2.5% less at 30 fps (195.7 versus 200.7 mJ). Only at 60 fps, where the slow pipelines saturate,

does the schedule decide energy again: 117–123 mJ for the pipelines that keep up, against 151–194 mJ for those that do not, measured per processed frame. This is the behavior described by Eq. (6). Using the idle power and the marginal energy measured on the backlog, Eq. (6) with the arrival rate substituted agrees with every below-capacity streaming cell within -0.2 to $+11.6\%$ (mean $+4.6\%$); the two slightly negative cells (the Ultralytics predictor at 15 and 30 fps, -0.2% each) lie inside the repeat spread of the backlog marginal estimate. For the three pipelines that saturate at 60 fps the arrival-rate substitution underestimates by 10–37%, because a queued pipeline never returns to the idle state the model assumes; substituting each pipeline’s measured processed-frame rate instead (as the published aggregates do) brings the error on those cells to 0.3–0.5%. The agreement below capacity comes mostly from the idle term, which dominates at camera rates. The marginal term alone is overestimated by 21–39% for the uint8 pipelines running below capacity; for the float-input sequential pipeline and the Ultralytics predictor (which normalizes on the GPU, Section 6) it agrees within -1.9 to $+3.4\%$. The clock gap between the backlog and the stream does not explain the uint8 gap: the worst cell (the event-synchronized sequential pipeline at 15 fps, $+39\%$) runs at 306 MHz on both. We cannot attribute the residue from these aggregates — CPU sleep-state behavior and any inter-frame GPU power gating differ between the two workloads, and the module rail does not separate them. Whatever its origin, the backlog marginal energy is an empirical upper envelope for the streaming marginal energy in every uint8 configuration we measured, and lies within -2 to $+4\%$ of it for the other two pipelines, not a prediction of it. At 15 and 30 fps the GPU runs at about 306 MHz in all pipelines, so the marginal energy per frame is lower than on the backlog, where the optimized pipelines run at about 1 GHz. In this regime the governor does what it is designed to do.

A corollary concerns clock locking. With `jetson_clocks`, the idle floor rises from 4.68 to 6.62 W. At 30 fps that is 221 mJ per frame before any work is done, already 8–13% more than the total energy of every default-governor pipeline in Table 10. For a camera application below its capacity, locking clocks is an energy cost without a throughput benefit.

Latency is set by the schedule. At low rates the overlapped schedule has the worst latency of all pipelines, despite having the highest backlog throughput. Its median latency at 15 fps is 85.3 ms, against 27.4 ms for the sequential pipeline and 26.8 ms for prefetch. Overlap finishes the rows of frame i only after frame $i+1$ has been decoded and enqueued, so every result waits about one inter-arrival period (66.7 ms at 15 fps). The penalty shrinks as the rate grows (51.9 ms at 30 fps, 35.5 ms at 60 fps). The sequential pipeline is the fastest per frame at 30 fps (median 21.6, p95 25.8 ms), and it keeps up with margin to spare (its backlog capacity of ≈ 50 images/s exceeds the 30 fps arrival rate by two thirds), but at 60 fps it fails. Plain prefetch keeps up at 60 fps, but its p95 rises to 99.3 ms. At 60 fps the governor runs prefetch at about 410 MHz, so its margin over the arrival rate is small and transient stalls build queues.

The sequential pipeline’s per-frame cost also depends on the arrival rate. Its median falls from 27.4 ms at 15 fps to 21.6 ms at 30 fps, although the work per frame is the same, and its CPU time per frame rises from 23.5 ms at 30 fps to 29.4 ms at 15 fps. The CPU clock was logged in these runs and does not explain this: the fastest core averages 1705 MHz at 15 fps and 1682 MHz at 30 fps. That statistic tracks the domain’s fastest core, not necessarily the core the waiting thread runs on, so a per-core effect remains possible — but as measured, the clocks give no support to the down-clocking mechanism of Section 6.3 here, and the extra CPU time per frame at the lower rate is at least as consistent with per-frame sleep/wake overhead.

The arrival-aware variant combines the two behaviors. When the next frame is not yet available, it finishes the in-flight frame instead of holding it. When frames queue up, it overlaps as before. Its p95 latency is within 7.4 ms of the best pipeline at every rate (p95 33.1, 33.2 and 40.6 ms at 15, 30 and 60 fps), it is the only pipeline whose p95 stays at or below 41 ms at all three rates, and its energy is within 0.4–4.3% of the full pipeline. Its predictions are byte-identical to the other uint8 pipelines.

Answer to RQ3. At camera rates below a pipeline’s capacity, energy per frame is dominated by the idle floor. The pipelines we tested differ by at most 2.5% at 15 fps and 4.2% at 30 fps. The schedule mainly determines latency and capacity margin. The schedule with the best backlog throughput has the worst latency at 15 fps, because it holds every result for one frame period. An arrival-aware schedule removes this penalty at a cost of 8.9% of backlog throughput and at most 4.3% of energy. Locking clocks raises the idle floor and is counterproductive in this regime.

8. RQ4: Model-level decisions under a fixed pipeline

8.1. Compression screening and a resolution router: negative results

Before the pipeline study, we spent most of our effort on the model: L1 and gradient-based channel pruning, mixed-precision QAT, distillation, and a per-image router that chooses between a 512 and a 640 engine from pre-inference statistics. Under a common short recovery budget (1,024 development images, five epochs, one seed) none of the compressed variants reduced compiled engine time at acceptable accuracy, and the router, which looked favorable on its fitting split, lost AP and small-object recall on an independent hash-verified verification split, so we rejected it. Appendix A reports both campaigns in full. These results do not show that compression cannot work. They show something narrower: for a 2.6 M-parameter NMS-free detector on this device and budget, the achievable engine-time savings are a small fraction of the $3\times$ governor effect of Section 5 — which is why the rest of this section varies resolution and model size while holding the optimization budget fixed.

Table 11

Other NMS-free detectors on all 5,000 val2017 images, 640, batch 1 (default: mean of two runs; locked: one run). “Naive” is float input with sequential execution and stream synchronization; “full” is uint8 input, two prefetch workers, overlap and event synchronization. MHz is the mean sampled GPU clock under the default governor.

Model	Pipeline	AP	AP _s	Default				Locked	
				img/s	mJ	W	MHz	img/s	mJ
YOLO26s	Ultralytics	0.4769	0.3020	25.1	263.5	6.62	306	63.0	194.4
	Ours, naive	0.4770	0.3019	24.5	271.6	6.66	306	52.2	221.8
	Ours, full	0.4769	0.3019	140.1	131.8	18.47	1019	140.4	133.3
YOLOv10n	Ultralytics	0.3836	0.1901	30.8	198.4	6.12	306	73.0	144.5
	Ours, naive	0.3837	0.1900	33.5	189.4	6.34	306	60.0	167.4
	Ours, full	0.3839	0.1901	212.4	76.9	16.33	1018	215.2	77.8

8.2. The value of a lower input resolution depends on the schedule

Lowering the input resolution is the cheapest model-level saving available. It needs no training, only a new engine. Fig. 7 compares static 512, 576 and 640 engines of the same pretrained YOLO26n checkpoint on all of val2017, in the naive pipeline (float input, sequential, stream sync) and in the full pipeline. Accuracy falls from 0.4026 AP (640) to 0.3864 (576) and 0.3748 (512). AP_s falls from 0.198 to 0.175 and 0.160. In the naive pipeline, 512 buys 27% more throughput (34.3 to 43.4 images/s) and 22% less energy per image. In the full pipeline the same saving shrinks to a few percent: 214.6, 222.2 and 218.7 images/s for 640, 576 and 512 (+3.5% and +1.9%; the repeat ranges of the lower resolutions lie above the 640 range, so the gain is small but consistent across the two repeats). The pipeline is now limited by the main thread (Section 6): at 1020 MHz the engine’s 4.2 ms sits just below the ≈ 4.7 ms per-image period the host stages impose, so most of the engine time saved by a smaller input is absorbed by the unchanged per-image host work.

Part of that saturation is Python. Re-running the same comparison with CPython’s cyclic garbage collector disabled (`gc.disable()`, identical predictions) raises all three throughputs to 229.9, 244.0 and 252.1 images/s, and the resolution spread widens to +6.1% (576) and +9.7% (512). Cyclic-GC pauses on the main thread absorb a further share of the saved engine time; with them removed, a lower input resolution still buys roughly a third of what it buys in the naive pipeline, not a negligible amount. The host work that remains — the main thread’s per-image period (Section 6) — depends on the source image and the pipeline, not on the network input. The energy benefit is robust either way: 73.7 to 58.0 mJ per image (21%) on a backlog with the default collector, and 71.2 to 55.8 mJ (22%) with it disabled, because the GPU finishes each inference sooner. The same 2.8 AP points therefore buy a large throughput gain in a naive pipeline and a much smaller one in an optimized pipeline — 2–10% depending on host-runtime hygiene. A compression or resolution decision evaluated in a naive pipeline, or in `trtexec`, does not carry over to an optimized one.

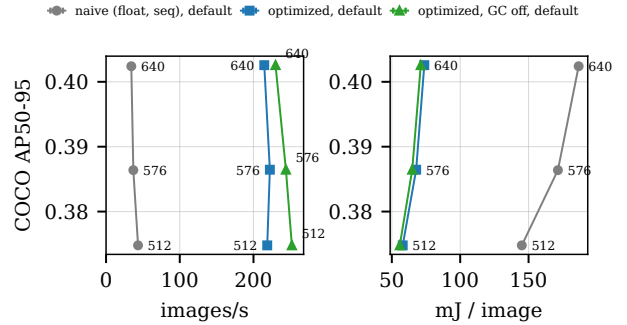


Figure 7: COCO val2017 AP against throughput (images/s) and energy per image (mJ) for static 512/576/640 YOLO26n engines, default governors, mean of two runs (whiskers: repeat min/max; labels: input size). In the full pipeline (squares), lower resolution buys only 2–4% throughput because the pipeline is limited by the main thread, not by GPU throughput (the GPU already runs at its maximum clock); disabling CPython’s cyclic GC (triangles, identical predictions) raises all three points and widens the spread to 6–10%.

8.3. Generality: YOLO26s and YOLOv10n

Table 11 repeats the three main configurations for YOLO26s (10.0 M parameters) and YOLOv10n. In every case the three pipelines agree in AP to within 4×10^{-4} , and the naive pipelines run the GPU at 306 MHz under default governors. YOLOv10n behaves like YOLO26n: 30.8 images/s with Ultralytics, 212.4 with the full pipeline (6.9 $\times$), at 76.9 instead of 198.4 mJ per image. YOLO26s has 3.9 $\times$ the parameters of YOLO26n and a 1.6 $\times$ longer engine time (6.78 versus 4.22 ms back-to-back), and its full pipeline is limited by the GPU rather than the CPU. It reaches 140.1 images/s (5.6 $\times$ Ultralytics), which is 95% of the 146.9 inferences/s that `trtexec` achieves back-to-back, and locking clocks does not change it (140.4). Under the default governors, the full YOLO26s pipeline outruns the Ultralytics YOLO26n predictor with locked clocks (140.1 vs. 73.9 images/s), at 0.074 higher AP and 7.5 less mJ per image (131.8 vs. 139.3). This comparison changes the pipeline and the model at once, so it does not show that the larger model is the better choice; YOLO26n in the same

full pipeline is faster still. It shows that the cost of the larger model is smaller than the cost of a naive pipeline, even when that pipeline runs with locked clocks. A deployment decision that compares model sizes in a naive pipeline therefore measures mostly the pipeline.

Answer to RQ4. Once the pipeline is fixed, model-level savings matter much less than the engine benchmark suggests. A 2.8-point AP sacrifice for a lower resolution yields 27% more throughput in a naive pipeline but only 2–4% in an optimized one (6–10% with cyclic GC disabled), plus lower energy on a backlog. Our compression variants did not reduce compiled engine time at acceptable accuracy within the budget screened. The router saved at most one eleventh of what scheduling saved, and it failed held-out verification. Whether the schedule effects carry over to other detectors and other accelerators is **RQ5** (Section 9); within this section, YOLOv10n and YOLO26s show speedups of 5.6–6.9× over the Ultralytics predictor, and for YOLO26s, whose engine is 1.6× slower than YOLO26n’s, the pipeline effect is large enough that the choice of pipeline matters more for throughput than the choice between the two model sizes.

9. RQ5: Cross-platform replication

The Jetson study above traces the in-app cost to one mechanism: an accelerator governor that reacts to the duty cycle the host pipeline creates, coupled to a CPU governor that reacts to how the host waits. How much of this is specific to that one board? We repeated the core experiments on two boards with different accelerator architectures (Table 12): a Raspberry Pi 5 whose Hailo-8 is a fixed-function INT8 NPU with on-chip NMS and no user-visible frequency control, and a Rubik Pi 3 (Qualcomm QCM6490), which we measure twice: on its CPU under a three-cluster schedutil DVFS, and on its Hexagon HTP NPU through a separately obtained QNN runtime. On both boards we kept the protocol of Section 4: official COCOeval on val2017 (all 5,000 images for accuracy; the ladder uses the first 1,000 on the slower Rubik), the same letterbox geometry and row conversion, hashed predictions, a rotated two-repeat campaign, and clock, thermal and (where available) power logging at 20 Hz on the Jetson; the Pi’s PMIC sustains 10 Hz and the Rubik sampler 13–18 Hz (clocks and thermals only, no power, Section 4). Two phases deviate from that template, and we flag them where they appear: the pinned-frequency probes are single runs per frequency, and the HTP streaming phase assigns the two rotated repeats one per governor, so each HTP streaming cell is a single run.

9.1. Raspberry Pi 5 with a Hailo-8 accelerator

The Hailo model-zoo YOLOv8n HEF offloads the whole network, including NMS, to the NPU; the host decodes, letterboxes into a uint8 NHWC buffer and converts the NMS output to COCO rows. The vendor benchmark (`hailortcli` benchmark) reports 431.6 images/s in throughput mode, under both CPU governors; the NPU clock is not user-controllable, and no register exposes it, so we cannot observe it directly.

The per-call NPU time we can measure includes host-side waiting inside the runtime; once that share is accounted for, the residual gives no sign of a duty-cycle response (below). A conventional sequential application on the same HEF reaches 65.7 images/s, 15% of the benchmark. The in-app gap exists on this accelerator too; with no observable accelerator clock we cannot rule in or out an internal NPU DVFS, but the mechanism we can identify is host-side, and the per-call NPU time, once its host-side wait share is accounted for (below), gives no sign of a duty-cycle response.

Table 13 shows the pipeline ladder. The mechanisms transfer. Prefetching decode nearly doubles throughput (1.86×), overlapping post-processing and adding a second decode worker gives another 1.5×, and a third worker adds nothing, the same saturation point as on the Jetson. In total the restructuring reaches 2.7× the sequential application (3.3× with the spinning wait, see below), with byte-identical predictions in every rung, repeat and governor (sha256 prefix 455ea22b07f1; official AP 0.320, AP_S 0.122 on all 5,000 images). The wait-policy effect also appears here, but with a different mechanism than on the Jetson: replacing the sleeping wait on the asynchronous inference handle with a spinning wait gains 22% under ondemand (177.9 to 217.5 images/s) and 29% under performance (169.9 to 219.8 images/s), where the CPU clock is pinned at 2.4 GHz. The gain therefore cannot be mediated by the CPU governor; it sits inside HailoRT’s wait and callback path, which we did not profile further. A separate governor effect exists but is smaller: a controlled probe (seq versus seq with a busy spinner) shows the governor holding 2.4 GHz instead of 2.2 GHz, worth 7.7% of throughput at 42% more energy. The NPU time itself, measured from submission to completion, stays at 6.3–9.3 ms across the whole ladder and does not respond to duty cycle; the wait time inside that interval absorbs the schedule differences. Energy per image falls 1.8× from sequential to the best variant. We stress that these millijoules cover only the SoC rails; the NPU’s own draw is on the unmonitored 5 V input. These values therefore compare SoC-side energy only: they do not establish whole-system energy rankings, because the unmeasured Hailo-8 energy can itself move with the schedule. Within-platform SoC-side comparisons remain meaningful between configurations that share the same engine and accuracy; cross-engine energy comparisons on this board need module-level power measurement. Streaming at 15, 30 and 60 fps is consistent with the Jetson’s idle-floor pattern: sequential and overlapped pipelines differ by 9% in energy per frame at 15 fps (102.0 versus 111.1 mJ, SoC side) and converge at 60 fps (37.4 versus 36.2 mJ). Two cautions apply: the two-repeat spread on the 15 fps overlapped cell reaches 19% (101–121 mJ), so that gap is indicative rather than established, and the sequential pipeline already sits at the idle floor within measurement error (the idle reading is a single 60 s window: 1.534 W ⇒ 102.3 mJ per frame at 15 fps).

Two differences from the Jetson matter. First, accuracy is not comparable across the boards: the HEF’s on-chip NMS

Table 12

Replication platforms. The Rubik Pi 3 is measured with two inference stacks: its CPU, and its Hexagon NPU.

	Raspberry Pi 5 + Hailo-8	Rubik Pi 3 (CPU)	Rubik Pi 3 (NPU)
SoC	BCM2712	QCM6490	QCM6490
CPU	4x A76, one domain, 1.5–2.4 GHz	8 cores, three domains, 0.8–2.7 GHz	(same)
Accelerator	Hailo-8 NPU (INT8), on-chip NMS	none used	Hexagon HTP V68 (INT8)
Model	YOLOv8n HEF	YOLO26n ONNX (1,300,6)	YOLO26n QNN ctx, (1,84,8400), host NMS
Runtime	HailoRT 4.23.0	ONNX Runtime 1.30.0	qai-appbuilder 2.50.40 ^b
CPU governor	ondemand, performance	schedutil, performance	
Power sensor	PMIC rails, SoC side only ^a	none usable ^c	

^aExcludes the 5V loads: the Hailo-8, fan and USB.^bPyPI wheel bundling libQnnHtp + V68 skel; the board’s QNN 2.46 predates the QAIRT 2.49.40-built context binary.^cThe board’s battery manager reports inconsistent values; we log clocks and thermals only.**Table 13**

Pipeline ladder on the Pi 5 + Hailo-8 (5,000 val2017 images, mean of two rotated repeats; p50 is the median per-image end-to-end pipeline latency; power is SoC-side only).

Governor	Rung	img/s	p50 ms	SoC mJ/img	CPU MHz
ondemand	seq	65.7	15.1	34.2	2020
	prefetch	122.0	16.1	25.5	2243
	full w1	149.8	14.6	23.0	2377
	full w2	177.9	25.2	25.2	2319
	full w2 spin	217.5	21.4	19.4	2300
	full w3	174.7	31.5	25.2	2327
performance	seq	74.6	13.3	34.8	2400
	prefetch	120.5	15.1	26.1	2400
	full w1	140.7	14.7	22.8	2400
	full w2	169.9	25.5	26.3	2400
	full w2 spin	219.8	21.2	20.0	2400
	full w3	166.4	32.1	27.2	2400

is compiled with a score threshold of 0.2, which truncates the low-score tail that COCO AP rewards; AP comes out at 0.320, against the 0.373 that Ultralytics reports for the same network at the conventional 0.001 threshold [40]. The threshold explains the direction and plausibly most of the magnitude, but without pre-threshold rows we cannot rule out smaller contributions from the HEF toolchain itself. Because every rung produces byte-identical predictions, this cancels in all within-platform comparisons. Second, the default ondemand governor costs less here than nvhost_podgov costs on the Jetson: it sits at 2.0 GHz even in the sequential pipeline and closes most of the gap to performance once the pipeline is overlapped (219.8 versus 217.5 images/s for the best rung). The Jetson’s threefold governor effect is specific to its aggressive GPU governor; what generalizes is that the benchmark number describes a duty cycle the application creates, not a property of the accelerator. One anomaly we cannot explain from clocks alone: on the overlap rungs the shipped ondemand governor is 5–6% faster than the pinned performance governor (177.9 versus 169.9 images/s at full_w2), while with the spinning wait the order inverts

(217.5 versus 219.8). NPU-side power states may differ between the two settings; we did not pursue it.

9.2. Rubik Pi 3: inference on the CPU

The Rubik Pi 3 runs the same YOLO26n ONNX graph as the Jetson float engine, but on the CPU (ONNX Runtime, default threads), so inference itself consumes the resource that the host stages and the wait policy also need. One structural difference from the accelerator ladders applies: in the overlap rungs the session call runs in each decode worker, so up to `max_workers` inferences execute concurrently and the *infer* column includes their mutual contention. The frequency probes of this subsection pin the run to the prime core (taskset, intra-op parallelism 1) so that a single frequency domain is exercised; the ladder runs use the default unpinned configuration. Accuracy first: on all 5,000 val2017 images the board reproduces the Jetson engine’s AP almost exactly (0.4027 versus 0.40257, AP_S 0.1977 versus 0.19766), so the model is a fixed point across the two stacks. CPU inference under multithreading emits rows in a nondeterministic order, so the byte-level hash does not apply directly; canonicalizing the rows (sorting by image, category, score and box) shows the predictions are in fact identical: all six rungs produce the same 143,362 canonical rows on the 1,000-image ladder subset in every repeat and governor (24 of 24 cells), and seq, full_w2 and full_w2_spin give identical AP (0.425).

Table 14 shows the ladder. Every effect that dominated on the accelerator boards shrinks or inverts. Prefetch alone no longer helps (−5%): the decode worker steals a core from the inference threads. Overlap with one worker gives the best result, but only +28% (versus 3.3× on the Pi and 6.3× on the Jetson, both under their shipped governors), and it nearly triples the median latency (304 to 896 ms) because the queued frame waits behind a 400 ms inference. The spinning wait, worth 20–22% on both accelerator boards, gains nothing here: a spinning thread competes with the inference threads for the same cores. The two governors differ by less than the thermal drift between identical repeats (3.26 versus 2.97 images/s for seq), a picture consistent with

Table 14

Pipeline ladder on the Rubik Pi 3 (CPU inference, 1,000 val2017 images of which 980 are timed after warm-up, mean of two rotated repeats with the repeat range in parentheses — two repeats cannot support an interval; p50 is the median per-image end-to-end latency, *infer* the ONNX Runtime call — in the overlap rungs the decode workers share the cores that inference uses, so the call itself lengthens; no power sensor on this board).

Governor	Rung	img/s	p50 ms	infer ms
schedutil	seq	3.26 (2.96–3.56)	304	254
	prefetch	3.11 (2.91–3.31)	641	269
	full w1	4.17 (3.99–4.36)	896	415
	full w2	3.85 (3.78–3.92)	1212	446
	full w2 spin	3.84 (3.69–4.00)	1237	451
	full w3	4.04 (3.92–4.17)	1415	426
performance	seq	2.97 (2.89–3.05)	329	275
	prefetch	2.96 (2.86–3.06)	671	281
	full w1	4.07 (3.89–4.26)	913	424
	full w2	4.00 (3.88–4.13)	1185	429
	full w2 spin	3.75 (3.63–3.87)	1262	461
	full w3	3.97 (3.84–4.10)	1443	434

both sitting at the thermal limit: the SoC reached 95.4–95.8 °C — the firmware ceiling — in every ladder run, and a repeat late in the campaign is up to 17% slower than an earlier identical one, a drift the Jetson never showed (median run-to-run range 0.8% there). We read no hardware throttle counter on this board, so we attribute the drift to thermal management by its signature (sustained ceiling temperature plus progressive slowdown) rather than by direct observation.

What remains controllable is the clock itself. Pinning the big core’s frequency domain to 806, 1862 and 2707 MHz moves the sequential pipeline through 1.7, 3.1 and 3.8 images/s; inference time scales from 575.7 to 255.2 ms, a 2.26× ratio for a 3.36× frequency ratio (one run per pin, and only the prime domain is pinned while the other two domains float, so we read this as a dose–response rather than a scaling law). On a CPU-inference device, frequency is a direct throughput control with no governor mystery: the pipeline never leaves the processor idle long enough to confuse *schedutil*. In the streaming workload (2, 5 and 10 fps) both pipelines keep up only at 2 fps, where the sequential variant again wins on latency (p50 290 versus 356 ms) and the big core drops to 2.3–2.5 GHz in the idle gaps. At 5 and 10 fps both fall behind their arrivals and the queue grows linearly: within a 300-frame stream the median latency reaches 23.9 and 39.3 s for the sequential pipeline, while the overlapped pipeline’s higher capacity (3.8 versus 2.8 images/s) holds it to 8.6 and 26.2 s, at a permanently saturated 2.7 GHz. (The streaming sequential capacity of 2.8 images/s sits below the ladder’s 3.26 because the streaming phase ran late in the campaign, when the board was hottest.) No schedule rescues a pipeline whose capacity is below the arrival rate; on this board the clock, or a cheaper model, is the only lever.

9.3. Rubik Pi 3 on its Hexagon NPU

The QCM6490’s intended accelerator is a Hexagon HTP (V68) NPU, which the CPU measurements above never touched: the board’s QNN runtime (2.46) predates the available INT8 context binary of the same YOLO26n graph (built with QAIRT 2.49.40), and the newer SDK requires a vendor account. We closed the gap with the *qai-appbuilder* 2.50.40 PyPI wheel, which bundles a matching *libQnnHtp* and V68 *skel*; with the DSP (digital signal processor) library path pointed at the wheel, the context runs unchanged. Two protocol differences follow from the context, not from our pipeline: it is INT8 rather than float, and it ends in the standard (1,84,8400) head, so NMS runs on the host with the reference val settings (confidence ≥ 0.001 , IoU 0.7, at most 300 detections), where the Hailo board sits at the opposite extreme with NMS fused into the HEF. The quantized stack scores AP 0.364 (AP_S 0.175) on all 5,000 images against 0.4027 (0.198) for the float CPU stack: eleven times the throughput of the best CPU pipeline under the same governor (43.6 versus 4.07 images/s under performance; 6.7× under *schedutil*) for 3.9 points of AP. One protocol caveat applies to that comparison: the context’s uint8 score encoding uses a quantization scale of 0.01045 (measured on the output grid), so the effective score floor is ≈ 0.01 rather than the reference 0.001 — the same mechanism we flagged for the Hailo, twenty-fold milder. It truncates only the far tail (on fewer than ten of the 5,000 images the stack emits no rows, all of them images whose float-stack detections score below 0.02). A direct sensitivity check quantifies this mechanism alone: quantizing the float stack’s scores to 0.01 steps and dropping the sub-0.01 tail lowers its AP from 0.4026 to 0.3999 (−0.27 points, AP_S −0.5), which is about 7% of the 3.9-point gap between the two stacks. The check exercises only the score floor — not the quantization of boxes, features or the two stacks’ different error distributions — so the remaining 93% of the gap lies elsewhere: the score floor is one measured contributor, not a bound on the full INT8 gap. Within the NPU stack, predictions are byte-identical across every ladder rung, repeat and governor (sha256 prefix 09de71d67bad on the ladder subset), and the accuracy run’s rows for those images match row for row. Each accuracy row comes from a single run; COCOeval is deterministic, so a repeat would differ only through runtime nondeterminism.

One deployment finding precedes the measurements: the bundled *appbuilder/QNN* userspace wedged the DSP session under concurrent multi-context inference, so our pipeline holds exactly one context and issues every NPU call from a single thread; overlap comes from pipelining decode, input fill and host NMS *around* the serialized NPU calls. Under that discipline the whole campaign ran without a single failed call. Separately, a harness bug in an early version of the evaluation script inflated every timed call by a per-call error print; we fixed it, reran the entire HTP campaign, and report only the corrected numbers. Both issues, and why the bug mattered most exactly where the paper looks at clock sensitivity, are detailed in Appendix C.

Table 15 shows the ladder, and the CPU board’s picture inverts back to the accelerator pattern. With a 24–28 ms NPU call, the host stages are again the tax, and the ladder recovers them: +24% under `schedutil` (22.9 to 28.5 images/s, the spinning-wait diagnostic; the best non-spinning rung reaches 27.8) and +52% under performance (28.8 to 43.6 images/s), where the best rung saturates the accelerator: its 22.9 ms period matches the effective per-call service time under load, marginally below the 24.4 ms isolated-call median. One consequence is that a second decode worker does not pay off under performance: `full_w2` is slower than `full_w1` (39.9 versus 43.6 images/s) and adds 39 ms of p50 latency, because NPU calls are serialized on one thread, so the extra worker cannot add accelerator parallelism and only lengthens the queue; under `schedutil` the two rungs tie (27.8 versus 27.7 images/s), the slowed feeder core being the binding constraint either way. That median corresponds to about 41 calls/s of back-to-back submission, and the best rung exceeds it (43.6 images/s), so the isolated call overstates the loaded service time — under sustained submission either queued calls overlap driver-side work or the loaded dispatch share shrinks; we cannot separate the two, and report both the isolated call and the saturated period. The governor, irrelevant on the CPU board, is here the largest single term: under `schedutil` the core that feeds the NPU settles at 0.95–1.0 GHz during the overlapped rungs and at 1.78 GHz in the sequential one — the less host work a rung leaves on the critical path, the lower the clock the governor chooses — and the same pipeline runs up to 57% faster under performance (43.6 versus 27.8 images/s on the matched `full_w1` rung; 44% on the matched `full_w2` rung, 39.9 versus 27.7 images/s). The pinned-frequency probe shows why the host clock matters so much for an “NPU” call: at 806, 1862 and 2707 MHz the sequential pipeline delivers 16.7, 24.0 and 28.6 images/s, and the measured NPU time itself falls from 35.8 to 24.4 ms. The three points are consistent with a decomposition into a fixed ≈ 20 ms NPU kernel plus a host-side dispatch term inversely proportional to the pinned core’s clock (least-squares fit; the mid point sits 0.8 ms above it, so the two-term form is descriptive rather than exact): a large share of each accelerator call is remote-procedure-call (RPC) work executed on the CPU. We stress that this is a descriptive decomposition, not a validated model — two parameters over three single-run pins leave one residual degree of freedom, and because the NPU exposes no observable clock or load register, the decomposition cannot distinguish a clock-invariant kernel from an accelerator DVFS whose response happens to be absorbed by the constant term. The governor invariance we report below is a hypothesis consistent with the data, not a measurement. The spinning wait gains nothing on this stack (38.2 versus 39.9 images/s under performance; it also shows the widest repeat spread of the campaign, 34.9–41.5 images/s across the two repeats — thermal drift, with the SoC climbing from 83 to 95 °C, not noise): with a single serial caller there is no asynchronous-handle wait for the spinner to heat. As on the Hailo, the NPU exposes no user-visible frequency control, and within

this decomposition the accelerator clock is not the moving part: what moves with the governor is the call’s host-dispatch share. The sequential call measures 24.4 ms under performance and 28.3 ms under `schedutil`, while the pinned probe’s kernel component (≈ 20 ms) is governor-invariant in the fit; correspondingly, under performance the best rung issues one call every 22.9 ms, the loaded per-call service time, while under the shipped governor the best rung issues one every 35.1 ms against a 28.3 ms sequential call, so the accelerator idles about a fifth of each period even when the queue never empties. (The *infer* column of Table 15 grows from 24.4 to ≈ 47 ms in the overlap rungs because there the measurement spans the queue wait behind the single serialized NPU caller, not because the accelerator slows down.) Streaming behaves as the capacities predict. Under `schedutil` the pipeline keeps up at 15 fps (p50 108 ms overlapped, 54 ms sequential) and queues linearly at 30 and 60 fps (p50 0.8 and 5.1 s overlapped, 2.4 and 6.7 s sequential, within a 480-frame stream). Under performance the overlapped pipeline sustains 30 fps at p50 47 ms while the sequential one (capacity 28.8 images/s) already falls behind (p50 0.39 s); at 60 fps both queue (p50 2.1 versus 4.7 s). Predictions agree row for row with the ladder on the shared images, in both campaigns. Each streaming cell here is a single run (the two rotated repeats were assigned one per governor), so the governor contrast is paired but not replicated.

Baking selection into the graph The Jetson and the Rubik CPU stacks run the NMS-free (1,300,6) head with selection fused into the model (the Hailo HEF fuses its own NMS on-chip instead), so for completeness we also built an end-to-end context for the HTP with the matching QAIRT 2.50.40 toolchain (INT8, 500 calibration images, the context binary generated on the board itself). One silent pitfall is worth recording: the exported head routes the box branch (range $\approx [-50, 800]$) and the score branch ($[0, 1]$) through a single concat–transpose–split, and per-tensor INT8 quantization gives the shared tensor one scale, which sets the score channel to zero at runtime. Calibration cannot fix this; graph surgery can — we transpose the two branches separately and expose boxes, scores and classes as three output tensors, so each receives its own scale (the V68 HTP rejects fp16 graphs outright, so mixed precision was not an option). The repaired context detects correctly and is faster end to end, because selection moves onto the DSP and the host NMS stage disappears: the same descriptive decomposition puts the fixed NPU kernel at ≈ 11.6 ms (endpoint and least-squares fits span 11.5–11.6 ms, with the mid probe point 0.3 ms above the fit) against ≈ 20 ms for the standard head, the accuracy run rises to 32.5 images/s and the best rung to 63.2 images/s (+45%), and the overlapped 30 fps stream cuts its p50 latency by 42% under performance (27 versus 47 ms). It also costs accuracy: AP 0.293 against 0.364, with the loss concentrated in small objects (AP_S 0.112 versus 0.175), where the in-graph INT8 box-decode chain (regression logits at 0.75 per step, decoded boxes at 3.3 px per step)

Table 15

Pipeline ladder on the Rubik Pi 3 with the Hexagon HTP NPU (INT8 context, host NMS; 1,000 val2017 images of which 980 are timed, mean of two rotated repeats with the repeat range in parentheses — two repeats cannot support an interval; NPU calls serialized on one thread, see text). *infer* is submission-to-completion of the NPU call; in the overlap rungs it spans the queue wait behind the single serialized caller, not pure accelerator time (the sequential rung gives the pure call: 24.4 ms under performance).

Governor	Rung	img/s	p50 ms	infer ms
schedutil	seq	22.94 (22.82–23.06)	43	28.3
	prefetch	25.60 (25.52–25.68)	77	28.8
	full w1	27.75 (27.75–27.76)	133	47.0
	full w2	27.68 (27.65–27.71)	173	46.2
	full w2 spin	28.45 (28.26–28.64)	173	53.2
	full w3	27.70 (27.63–27.77)	209	46.7
performance	seq	28.75 (28.67–28.83)	35	24.4
	prefetch	31.86 (31.74–31.98)	62	24.4
	full w1	43.59 (43.56–43.61)	89	40.0
	full w2	39.89 (39.54–40.24)	128	45.3
	full w2 spin	38.22 (34.92–41.53)	135	47.2
	full w3	40.01 (37.47–42.56)	146	42.5

hurts most (Table 16). We therefore keep the standard head with host NMS as the primary HTP stack and report the fused head as a measured configuration comparison: on this toolchain, the fused-selection configuration runs 45% faster at peak and scores 7 AP points lower than the standard-head configuration. The comparison is not a clean ablation — the two contexts come from different QAIRT releases (2.49.40 versus 2.50.40) and different calibration sets — so the -7 AP is the observed difference between these two configurations, not an isolated cost of fusing selection.

9.4. What transfers, and what does not

Across three boards and four inference stacks the same experiment gives four shades of one answer. The *benchmark-versus-application gap* appears on every stack we measured: 431.6 images/s versus 65.7 images/s on the Pi 5 (15%), 4.2 ms versus 13.2 ms on the Jetson, and on the Rubik a 254 ms inference inside a 304 ms sequential loop on the CPU and, under the shipped governor, a 28 ms NPU call inside a 43 ms loop on the HTP. These four gaps are not all of the same kind: each compares an isolated engine or inference-call benchmark with a sequential application, but the content of the difference is stack-specific. On the Jetson it is mostly the governor’s response to the application’s duty cycle (Section 5); on the Pi, where the accelerator call is fixed-cost, it is the host work around that call; on the HTP it additionally contains the host-dispatch share of each accelerator call and that share’s response to the CPU governor; on the Rubik CPU it is bounded by thermal saturation. The *host-pipeline ladder* helps on all four stacks, but its size depends on what else moves with the schedule (best rung versus sequential, each under the board’s shipped governor): 6.3 $\times$ on the Jetson against the naive sequential rung (float input, stream synchronization; 4.3–5.3 $\times$ against the stronger

sequential variants of Table 6; the 2.4 $\times$ headline of Section 1 uses the locked-clock baseline), where overlapping also lifts the GPU out of its DVFS floor, so the ladder undoes both the host-stage tax and the clock collapse; 3.3 $\times$ on the Pi, whose sequential pipeline is the most host-bound of the four; 1.3 $\times$ on the Rubik CPU; and only 1.2 $\times$ on the HTP under the shipped governor, where the CPU governor settles the core that feeds the DSP near 1 GHz as soon as the pipeline overlaps — fixing the governor first re-opens the ladder to 1.5 $\times$ (performance column of Table 15). The *wait-policy effect* appears on both asynchronous-handle accelerators but at different rungs and through different mechanisms. On the Jetson it is a property of the schedules that actually wait: replacing the sleeping wait gains 20–25% on the sequential and prefetch rungs, mediated by the CPU governor (the busy-loop and uclamp experiments of Section 6.3), and nothing on the full two-worker pipeline, whose host never goes idle on the critical path. On the Pi the same swap gains 22–29% on the *best* rung (full_w2), and it persists at a pinned CPU clock, so it lives in the runtime’s wait path rather than the governor. It vanishes where no such wait exists: when the CPU itself computes, because the spinner steals cycles from the inference it waits for, and on the HTP stack, where the single serial caller waits on a bounded queue rather than an async handle. The *accelerator governor*, the Jetson’s dominant term, is observable only where the accelerator exposes a clock: neither NPU offers a user-visible clock or load register, so an internal accelerator DVFS cannot be ruled out — but it is not needed to explain these measurements: the per-call NPU time we can measure includes the host-side wait and queueing inside the runtime (it grows from 6.3 to 9.3 ms across the Pi’s ladder, and the HTP’s infer column spans queue waits), and once that share is accounted for, neither stack shows a residual that moves with the host duty cycle — on the HTP the overlap rungs saturate the loaded per-call service time under performance (under the shipped schedutil, whose slowed core stretches the call’s dispatch share, the accelerator still idles about a fifth of each period at the best rung; the *infer* column of the ladder tables grows in the overlap rungs because it spans queueing behind the single serialized caller, not because the accelerator slows down; Section 9.3). But the governor story does not disappear on the NPU boards, it moves: on the HTP stack the CPU governor becomes the largest term (57% on the matched full_w1 rungs), because schedutil reads the idleness the accelerator creates and down-clocks the very core that feeds the DSP, and every “NPU” call carries a host dispatch share that grows as the core clock falls. Finally, sustained CPU inference introduces a further constraint: the SoC sits at its 95 °C ceiling and otherwise identical repeats separate measurably. The NPU stacks approach the same ceiling under performance (91–95 °C). The Pi shows no drift, but on the HTP the two heaviest performance-governor overlap rungs do: full_w2_spin falls from 41.5 to 34.9 images/s between repeats as the SoC climbs from 83 to 95 °C, and full_w3 from 42.6 to 37.5, so those two cells mix the governor effect with a thermal one (the schedutil

Table 16

Head configuration comparison on the Rubik HTP: the standard head with host NMS versus the NMS-free head fused into the INT8 context. This is not a clean ablation — the two contexts come from different QAIRT releases (2.49.40 versus 2.50.40) and different calibration sets — so the accuracy difference is a property of the two configurations, not an isolated cost of fusing selection. Accuracy on all 5,000 val2017 images (same pipeline, serialized NPU calls); throughput is the accuracy run and the best ladder rung per governor. Under `schedutil` both best cells are the spinning-wait diagnostic (best non-spinning rungs: 27.8 and 36.9 images/s); under performance the fused head’s spin and non-spin best rungs tie within run-to-run noise (63.2 versus 63.0).

Head	AP	AP ₅₀	AP _S	acc. img/s	best img/s (sched/perf)
standard (1,84,8400), host NMS	0.364	0.527	0.175	22.8	28.5 / 43.6
NMS-free, fused (3 outputs)	0.293	0.477	0.112	32.5	39.6 / 63.2

rows and both sequential ladders show no drift). On the CPU stack the thermal limit is the binding constraint. And *streaming behavior* follows capacity on the NPU boards exactly as Section 7 predicts (Table 17): the Pi keeps up at every rate under either pipeline — its sequential capacity of 65.7 images/s already exceeds 60 fps — while the HTP queues precisely when the arrival rate passes the capacity the ladder measured. One latency mechanism does not transfer literally. The Jetson’s plain overlap holds each frame’s rows until the next frame is enqueued, adding about one arrival period (Section 7); that hold is a property of our double-buffered host loop, and it is what the arrival-aware variant removes. Neither NPU stack has it: HailoRT delivers each frame’s results through an asynchronous completion callback, so the Pi’s overlapped p50 tracks the service time (≈ 17 ms) at every rate, and the HTP pipeline submits rows as soon as the serialized call returns. There the shipped governor is what inflates latency: the slowed feeder core stretches both the call and the host NMS that overlaps it, so at 15 fps the overlapped p50 is 108 ms under `schedutil` against 42.6 ms under performance. What transfers unchanged is the capacity-margin behavior: queues build exactly when the arrival rate passes the ladder-measured capacity. For a practitioner the ordering of Section 10 survives with two amendments: on CPU-inference devices, stop after decode overlap with a single worker and spend the effort on clocks and cooling; on NPU devices, check the CPU governor early — on the QNN/HTP stack it was the largest single term, while the Hailo stack’s shipped governor was already within 1% of the pinned one at the best rung.

Answer to RQ5. The findings carry over in outline, not in mechanism. Across all four stacks (Jetson TensorRT, Hailo-8, Rubik CPU, Rubik HTP), isolated engine or inference-call measurements understate the end-to-end application cost, but for different reasons: on the Jetson the gap is a GPU-governor-induced engine slowdown; on the Pi it is host and runtime overhead around a fixed-function NPU; on the Rubik CPU it is shared CPU resources and thermal saturation; and on the HTP it is the CPU-side dispatch share of each call and that share’s response to the CPU governor. In each case the host-pipeline ladder recovers throughput without touching the engine, and streaming queues exactly when the arrival rate passes the ladder-measured capacity. Across model architectures (Section 8.3), YOLOv10n and YOLO26s (whose engine

takes 1.6× longer per image) reproduce the same schedule effects (5.6–6.9× over the Ultralytics predictor), so the conclusions are not specific to YOLO26n.

10. Discussion

10.1. Implications for benchmarking

A back-to-back engine benchmark measures the GPU at a clock that a sequential application never reaches on this device. Engine-level speedups therefore answer a different question from the one a deployment asks. The gap is not a constant factor. It depends on the schedule (Table 6), the arrival rate (Table 10) and the model’s own duty cycle (YOLO26n versus YOLO26s in Table 4). In our experiments it is not even monotone in the arrival rate: the median latency of the uint8 and float sequential pipelines is higher at 15 fps than at 30 fps, and their CPU time per frame is higher as well, which points to a lower CPU clock during longer idle gaps (Section 6.3). We therefore suggest that efficiency claims for edge detectors include at least: (i) an end-to-end decode-to-detections measurement, (ii) the GPU and CPU governors and the power mode, (iii) the GPU and CPU clocks logged during the measurement, (iv) how the harness stores results, and (v) the workload scenario (backlog, fixed-rate stream or single frame), in the spirit of the MLPerf scenarios [28]. Locking clocks is a useful diagnostic. It is not a neutral benchmarking choice, because it removes exactly the interactions that dominate behavior under the stock governors a device ships with. Under locked clocks the synchronization call looks irrelevant on the sequential and prefetch schedules, while under the default governors it costs a fifth of the throughput; and on the two-worker schedule only the synchronization boundary matters (24% for stream synchronization), identically in both regimes (Section 6.3). Locking also raises idle power by 1.9 W, which at camera rates costs more energy than any pipeline choice saves (Section 7).

10.2. Guidelines for practitioners

The ordering that worked on this platform was:

1. Take host stages off the GPU’s critical path first (decode prefetch, overlapped post-processing), and choose a wait that does not let the CPU governor lower the host’s clock (a spinning wait, or a per-task utilization clamp, which needed no root privileges on

Table 17

Streaming on the NPU boards: p50 latency per frame in ms (release to finished rows; 1,000-frame streams on the Pi, 480-frame streams on the HTP). Pi: mean of two rotated repeats under the shipped `ondemand` governor, with SoC-side energy per frame in parentheses. HTP: one run per cell and governor (the two rotated repeats were assigned one per governor), so the governor contrast is paired but not replicated; the fused-head variant is discussed in the text. Queueing cells are rounded to two significant figures.

Board	Pipeline	Governor	15 fps	30 fps	60 fps
Pi 5 + Hailo-8	seq	ondemand	17.4 (102.0 mJ)	16.9 (61.0 mJ)	15.7 (37.4 mJ)
	full w2	ondemand	17.3 (111.1 mJ)	17.0 (58.7 mJ)	15.6 (36.2 mJ)
Rubik HTP	seq	schedutil	54.2	2400	6700
		performance	35.2	390	4700
	full w2	schedutil	108	820	5100
		performance	42.6	47	2100

our system and costs less energy; Section 6.3). Use the clamp when a sleeping wait induces frequency collapse; once the pipeline keeps the host busy, the wait policy no longer matters, but the synchronization boundary does — synchronize on per-frame events, not the whole stream, in an enqueue-ahead loop (Table 9).

2. Move preprocessing into the engine once the CPU becomes the bottleneck. Its value grows from 18% to 2.4× as the schedule improves.
3. Match the schedule to the workload. Use overlap with several decode workers for backlogs and high rates. Use the arrival-aware variant when frames arrive at camera rates and latency matters. Do not lock clocks for a camera application that runs below capacity.
4. Only then evaluate model-level savings (resolution, pruning, quantization), and evaluate them in the optimized pipeline, with application-level metrics.

Step 4 matters because the pipeline effect is larger than the model effect. In our data the full YOLO26s pipeline under default governors is faster and more accurate than the Ultralytics YOLO26n predictor with locked clocks (Table 11). Model sizes should be compared within the same, optimized pipeline.

10.3. Why compression is not useless

The negative compression results are specific to a 2.6 M-parameter detector whose compiled engine takes about 4 ms, on a device whose host stages cost more than the engine at full clock. For larger models, where the engine is the bottleneck even in an optimized pipeline (YOLO26s reaches 95% of its back-to-back engine rate), reducing engine work translates directly into throughput, and compression regains its usual role. Our results argue for evaluating compression in the regime where it will be deployed, not for abandoning it.

10.4. Relation to frequency-control research

DVFS research on edge inference usually controls the frequency directly [15–17]. Our results are complementary. With the stock governor left in place, the application’s

own schedule determines the operating point, and a well-scheduled pipeline lets the stock governor choose the top clock on its own. A governor-aware design could go further. For example, at a known frame rate it could deliberately run at the lowest clock that meets a latency target. The 60 fps streaming results show the potential: the governor ran the optimized pipelines at 405–430 MHz, and their marginal energy per frame fell below the backlog value. Our CPU results add a second dimension. The CPU and GPU governors act independently, but through the host pipeline the CPU clock sets the GPU’s duty cycle and therefore its clock (Section 6.3). A coordinated policy would have to treat the two as one control problem. Utilization clamping is a first, coarse step in this direction: it lets the application tell the CPU governor that its sleeping thread sits on the critical path of the GPU.

11. Threats to validity

Internal validity. Scheduling variants are compared at byte-identical predictions, so accuracy is not a confound. Timing confounds we controlled are run order (rotated), thermal state (10 s pauses; junction temperature logged; on the Jetson, maximum 67.4 °C in application runs and 69.2 °C in the `trtexec` sweep, far from throttling — the Rubik CPU runs, by contrast, sit at the 95 °C thermal limit and show a quantified 17% drift, Section 9.2), page-cache state (warm reads before each phase) and clock state (stored and restored, verified after each phase). Remaining confounds are the small number of repeats per cell and the 20 Hz sampling of power and clocks, which cannot resolve sub-50 ms clock changes (Section 5). The memory (EMC) clock has its own governor. A direct `devfreq` lock attempt did not hold on this release — a silent-failure mode documented independently on the same board family [21] — so in the fixed-frequency sweep, where only the GPU domain is pinned, we logged the memory clock and bounded its influence to 2% of the fitted slope (Section 5). `jetson_clocks` does hold it (verified by `readback`, Section 4), so the locked-versus-default comparisons include any memory-clock effect. Jetson campaign runs last 9–203 s, and the cross-board accuracy sweeps run longer (the Rubik CPU sweep takes ≈26 min); we did not

test deployment-length steady state, and the ramp visible in Fig. 3 means the first seconds of a run differ from the rest. The Ultralytics baseline uses its own engine export. Its AP agrees with ours to 3×10^{-5} , but engine tactics may differ slightly.

Construct validity. `VDD_IN` is the module’s main input rail, not whole-system power: it excludes carrier-board loads such as the fan, USB peripherals and storage regulators. It still overstates the energy of the SoC compute rails alone, and the idle cost of locking clocks includes the higher memory clock as well as the CPU and GPU clocks. We did not validate the on-board sensor against an external power meter; our comparisons rely on its consistency between runs rather than on its absolute accuracy. Our “latency” starts when a JPEG file is available to the pipeline; decode is part of the measured work. Camera capture, image-signal-processor (ISP) processing and output transmission are excluded. Process CPU time is reported per image and includes all threads of the process.

External validity. The main study uses one Jetson Orin Nano Super with one software stack; Section 9 partially offsets this with a replication on a Hailo-8 NPU board and a Qualcomm board measured on both its CPU and its Hexagon NPU, but other Jetson modules, power modes, JetPack releases, hardware JPEG decode [25] or DeepStream pipelines may shift the magnitudes, and other vendors’ governors may behave differently. On the Pi 5 the Hailo-8 sits on the unmonitored 5 V rail, so Pi energies cover the SoC only, and its HEF’s on-chip NMS uses a 0.2 score threshold, so Pi accuracy is not comparable with the other boards. On the Rubik Pi 3 there is no usable power sensor; the NPU stack runs through a third-party bundled runtime (`qai-appbuilder`) rather than the vendor SDK, and its primary INT8 context ends in the standard head with host-side NMS (a fused NMS-free head was measured as a configuration comparison and costs 7 AP on this toolchain, Section 9.3), so its accuracy is not comparable with the float stacks; the QNN userspace wedged under concurrent inference calls, so our NPU pipeline serializes accelerator calls and overlaps host stages only; and multi-threaded CPU inference emits rows in a nondeterministic order, so prediction equivalence on the CPU stack is shown on canonicalized (sorted) rows — identical in all 24 ladder cells — rather than by a byte-level hash. Our pipelines are written in Python. A native C++ or DeepStream pipeline would shorten the host stages and could reach the GPU-bound regime with fewer decode workers. We also used batch size 1 throughout, as a camera application would. Batch size and GPU frequency are known to interact [14]; we did not measure batching. The compression screening used a short recovery schedule on a development split with one seed and `opt0` TensorRT builds. It is evidence about that budget, not a general verdict on pruning or quantization.

12. Conclusion

For NMS-free detectors on an embedded GPU, the deployed cost of a model is not a property of the model alone. Under the stock governor, the same YOLO26n engine costs 4.2 ms or 13.2 ms depending on how busy the application keeps the GPU. A simple duty-cycle model, with inverse-clock engine time, a load-band governor and an idle-plus-marginal energy split, describes the measurements. Restructuring only the host pipeline, without locking clocks and at unchanged COCO accuracy, raised throughput 2.4× over the best sequential pipeline with locked clocks and 6.9× over the standard predictor, and cut energy per image 2.6× relative to the latter. The CPU governor takes part in the same loop: a sleeping CUDA wait lowers the host’s clock and, through the host, the GPU’s. A per-task utilization clamp, set without root privileges, recovers most of this governor-mediated loss; on the full two-worker schedule the wait policy itself costs nothing, and only the synchronization boundary (24% for stream synchronization) remains, at locked and default clocks alike. At camera rates below the pipeline’s capacity the picture changes again: idle power then sets energy per frame, and the schedule mainly sets latency — until the rate approaches capacity, where the schedule decides energy once more (117–123 mJ per processed frame for the pipelines that keep up at 60 fps, 151–194 mJ for those that do not). The right schedule therefore depends on the arrival rate. The same restructuring changed the value of every model-level decision we examined. Lower resolution gave only 2–4% throughput in the optimized pipeline (6–10% with the garbage collector disabled), against 27% in the naive one. Compression and routing gave less than scheduling. The pipeline effect was larger than the step from YOLO26n to the 1.6× slower YOLO26s. On a Raspberry Pi 5 with a Hailo-8 NPU the same host restructuring gives 2.7× (3.3× with the runtime’s wait policy replaced) with identical predictions, even though the accelerator exposes no frequency control, while on a Qualcomm board the ladder flattens to 1.3× on the CPU — where thermal management becomes the binding constraint — and on its Hexagon NPU the CPU governor becomes the largest term, because the scheduler down-clocks the core that feeds the accelerator. We hope that these results and the accompanying data encourage edge-detection research to report end-to-end, governor-aware measurements alongside engine benchmarks.

A. Compression screening and router details

Before the pipeline study, we spent most of our effort on the model. Table A.1 summarizes the screening. All of it was done on a 1,024-image development split of COCO train2017, disjoint from val2017, with official COCOeval. Each method was given the same short recovery budget: 1,024 training images, five epochs and one seed.

Channel pruning removed 25% of the output channels of four backbone convolutions, 5.6% of the parameters. Our object-risk criterion scores channels by gradient-weighted foreground activations and a fake-INT8 perturbation proxy, and it kept 2.7 AP points more than L1 magnitude pruning at the same budget. Both pruned models still lost 6.8–9.5 AP points. Neither was faster after compilation, because the pruned early convolutions are not where TensorRT spends its time. Ultralytics QAT produced a mixed-precision engine (about half of the convolutions in INT8, the rest FP16/FP32 behind reformatting layers). It was slower than FP16 under both builder levels. Distillation from the fine-tuned teacher reduced AP in both pruning settings (L1: 0.369 to 0.338; object-risk: 0.395 to 0.373 on the development split). We also pruned the parts that per-layer profiles showed to be expensive (P3/P4 classification-head channels and the attention block’s feed-forward width). We found no stable change in engine time beyond run-to-run noise (about 6.5 ms back-to-back with opt0 builds; the table’s 21.8 ms column is the same engine inside a sequential application, and the difference between the two is the governor effect of Section 5). These results come from a short recovery budget — one seed, five epochs, opt0 builds on one device. With longer training, the pruned models would likely recover more AP, and a full opt3 build sweep might redistribute the engine time differently. They do not show that compression cannot work. They show something narrower: for a 2.6 M-parameter NMS-free detector on this device and budget, the achievable engine-time savings are a small fraction of the 3× governor effect of Section 5.

A.1. The per-image resolution router

We fit a logistic router on cheap pre-inference image statistics. It chooses a 512 or 640 engine per image, and its threshold was fixed to send 20% of images to 640. On its 512-image held-out half, it slightly beat a fixed 576 engine in both AP and median time. We then froze it and evaluated it once on an independent, hash-verified 1,024-image train2017 split that played no part in fitting or threshold selection (Section 4). Against fixed 576 it lost 0.0089 AP (0.4386 vs. 0.4476) and 0.0147 AP_S, and both 512-image halves lost AP. Its median application time was 1.1–1.4 ms lower across three paired runs (41.3–41.9 versus 42.4–43.2 ms), but its 95th-percentile time was not. Small-object recall at IoU 0.5 fell by 1.55 points (paired image bootstrap 95% interval [−2.98, −0.19]). We therefore rejected it. In the same units, the scheduling steps of Table 6 save 15–28 ms of median per-image latency depending on the starting pipeline; the router’s ≈1.2 ms is at most one eleventh of that. The comparison is in latency rather than

Table A.1

Compression screening for YOLO26n (development split of 1,024 train2017 images, one seed, five recovery epochs, TensorRT 10.3). The two blocks come from different measurement contexts and are not directly comparable. Block (a): median host-measured copy+execute+copy per image in a sequential application with opt0 builds, which includes the down-clocking of Section 5. Block (b): back-to-back trtexec GPU compute with opt3 builds (driver keeps the GPU near 1,020 MHz; AP_S not recorded in that harness).

Variant	AP	AP _S	ms
<i>(a) sequential application, opt0 build</i>			
FP16, fine-tuned (reference)	0.4691	0.2164	21.8
Mixed QAT (50/99 conv INT8)	0.4532	0.2038	24.4
L1 channel pruning (−5.6% params)	0.3741	0.1426	21.8
Object-risk pruning (−5.6% params)	0.4007	0.1613	21.8
L1 pruning + QAT	0.3612	0.1372	24.3
Object-risk pruning + QAT	0.3924	0.1594	24.5
<i>(b) back-to-back trtexec, opt3 build</i>			
FP16	0.4686	–	4.07
Mixed QAT	0.4593	–	5.00

period: the router was evaluated in a sequential pipeline, so its saving does not compose with an overlapped schedule.

B. Streaming warm-up anchoring

An earlier version of the streaming harness anchored the arrival clock before the harness’s internal warm-up, so the first timed frames inherited the warm-up backlog: the Ultralytics predictor’s 30 fps p95 read 447 ms. With arrivals re-anchored at the warm-up boundary — the numbers reported in Section 7, with predictions unchanged — the same cell’s p95 is 188 ms: more than half of that tail was start-up bias. We report the re-anchored numbers throughout.

C. QNN userspace notes

Two findings about the bundled qai-appbuilder/QNN userspace (QAIRT 2.50.40 wheel on QCM6490, V68 HTP) shaped the HTP campaign. First, it is not safe under concurrent inference: issuing calls from two contexts at once wedges the DSP session after a few hundred calls (an empty output or a session that never returns; retries and context rebuilds do not recover it). We can only speak for this bundled userspace as shipped — the vendor SDK advertises multi-context support, which we did not exercise. Our pipeline therefore holds exactly one context and issues every NPU call from a single thread; overlap comes from pipelining decode, input fill and host NMS *around* the serialized NPU calls. Under that discipline the whole campaign ran without a single failed call.

Second, one harness bug is worth recording. An earlier version of the evaluation script passed the input buffer to the appbuilder wrapper unflattened, and the wrapper’s reshape helper caught the resulting broadcast error and printed it on every call, inside the timed region (≈1 ms per call at

the shipped governor, more at pinned low clocks; predictions were unaffected — the native call received the correct buffer). We fixed the call, reran the entire HTP campaign, and report only the corrected numbers; the fix moved the shipped-governor rungs by ≈ 1 ms per call, the performance ladder by up to 13% (the pinned core now spends its cycles on dispatch instead of printing), and the 806 MHz probe point by 8 ms per call, so it mattered most exactly where the paper looks at clock sensitivity.

Data and code availability

The measurement scripts (pipeline, Ultralytics baseline, power sampler, campaign runner, aggregation and figure generation), the raw timestamped traces (Jetson: power, clocks, GPU load and temperature at 20 Hz; Pi 5: PMIC power at ≈ 10 Hz and clocks; Rubik: clocks at 13–18 Hz, no power rail), per-run reports, SHA-256 hashes of all prediction files, and the COCOeval summaries, and the aggregation and plotting scripts that generate the tables and figures from these traces are provided at <https://github.com/iamdinhthuan/governor-is-part-of-the-model>, whose README maps every table and figure to its data and generator script, so nearly every reported number can be recomputed; a small number of numbers remain verbatim in the LaTeX source (the Table A.1 screening table and the prose router results, whose aggregators are included). The complete raw campaign data, including the 1.6 GB (uncompressed) of prediction files whose SHA-256 hashes anchor the byte-identical-prediction claims, are archived on Zenodo [41].

CRedit authorship contribution statement

Dinh Thuan Nguyen: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing—original draft. **Lam Phuong Nguyen:** Validation, Data curation, Writing—review and editing. **Vinh Huy Nguyen:** Methodology, Visualization, Writing—review and editing. **Sy Vu Quang:** Validation, Writing—review and editing. **Mohan Rajesh Elara:** Supervision, Writing—review and editing. **Anh Vu Le:** Supervision, Project administration, Writing—review and editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Factory Droid to run the measurement campaigns and assist with drafting. The authors reviewed and edited all content and take full responsibility for it.

References

- [1] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, Microsoft COCO: Common objects in context, in: European Conference on Computer Vision (ECCV), 2014, pp. 740–755.
- [2] H. Li, A. Kadav, I. Durdanovic, H. Samet, H. P. Graf, Pruning filters for efficient ConvNets, in: International Conference on Learning Representations (ICLR), 2017.
- [3] G. Fang, X. Ma, M. Song, M. B. Mi, X. Wang, DepGraph: Towards any structural pruning, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023.
- [4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.
- [5] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531 (2015).
- [6] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, G. Ding, YOLOv10: Real-time end-to-end object detection, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.
- [7] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, M. E. Kalfaoglu, Ultralytics YOLO26: Unified real-time end-to-end vision models, arXiv:2606.03748 [cs.CV] (2026).
- [8] Ultralytics, Ultralytics YOLO26 documentation, <https://docs.ultralytics.com/models/yolo26/>, software release 8.4.172 used in this work (2026).
- [9] The Linux Kernel Documentation, Utilization clamping, <https://docs.kernel.org/scheduler/sched-util-clamp.html>, accessed 2026-10-05 (2024).
- [10] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, End-to-end object detection with transformers, in: European Conference on Computer Vision (ECCV), 2020, pp. 213–229.
- [11] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, J. Chen, DETRs beat YOLOs on real-time object detection, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024.
- [12] L. L. Zhang, S. Han, J. Wei, N. Zheng, T. Cao, Y. Yang, Y. Liu, nn-Meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices, in: ACM International Conference on Mobile Systems, Applications, and Services (MobiSys), 2021.
- [13] Z. Tang, Y. Wang, Q. Wang, X. Chu, The impact of GPU DVFS on the energy and performance of deep learning: An empirical study, in: Proceedings of the Tenth ACM International Conference on Future Energy Systems (e-Energy), 2019.
- [14] S. M. Nabavinejad, S. Reda, M. Ebrahimi, Coordinated batching and DVFS for DNN inference on GPU accelerators, IEEE Transactions on Parallel and Distributed Systems 33 (10) (2022).
- [15] S. Kim, K. Bin, S. Ha, K. Lee, S. Chong, zTT: Learning-based DVFS with zero thermal throttling for mobile devices, in: ACM International Conference on Mobile Systems, Applications, and Services (MobiSys), 2021.
- [16] Y. Han, Z. Nan, S. Zhou, Z. Niu, DVFS-aware DNN inference on GPUs: Latency modeling and performance analysis, arXiv preprint arXiv:2502.06295 (2025).

- [17] Z. Zhang, Z. Wu, J. Liu, L. Mottola, SparseDVFS: Sparse-aware DVFS for energy-efficient edge inference, arXiv preprint arXiv:2603.21908 (2026).
- [18] A. Pathania, Q. Jiao, A. Prakash, T. Mitra, Integrated CPU-GPU power management for 3D mobile games, in: Proceedings of the 51st Annual Design Automation Conference (DAC), 2014. doi: 10.1145/2593069.2593151.
- [19] M. Karzhaubayeva, A. Amangeldi, J.-G. Park, CNN workloads characterization and integrated CPU-GPU DVFS governors on embedded systems, IEEE Embedded Systems Letters 15 (4) (2023) 202–205. doi:10.1109/LES.2023.3299335.
- [20] Y. Gong, Y. Wu, Z. Zhan, P. Zhao, L. Liu, C. Wu, X. Tang, Y. Wang, Lotus: Learning-based online thermal and latency variation management for two-stage detectors on edge devices, in: 61st ACM/IEEE Design Automation Conference (DAC), 2024. doi:10.1145/3649329.3657310.
- [21] J. Kang, Edge-inference governors need memory-clock state, arXiv:2606.16106v3 [cs.PF] (2026).
- [22] S. Mittal, A survey on optimized implementation of deep learning models on the NVIDIA Jetson platform, Journal of Systems Architecture 97 (2019) 428–442. doi:10.1016/j.sysarc.2019.01.011.
- [23] R. Hadidi, J. Cao, Y. Xie, B. Asgari, T. Krishna, H. Kim, Characterizing the deployment of deep neural networks on commercial edge devices, in: Proceedings of the IEEE International Symposium on Workload Characterization (IISWC), 2019.
- [24] D. Richins, D. Doshi, M. Blackmore, A. T. Nair, N. Pathapati, A. Patel, B. Daguman, D. Dobrijalowski, R. Illikkal, K. Long, D. Zimmerman, V. J. Reddi, AI tax: The hidden cost of AI data center applications, ACM Transactions on Computer Systems 37 (1–4) (2021) 3:1–3:32. doi:10.1145/3440689.
- [25] T. Chen, N. Alnaasan, J. Yao, A. Shafi, H. Subramoni, D. K. Panda, Accelerating data preprocessing for efficient vision model inference on Jetson edge device, arXiv preprint arXiv:2608.27655 (2026).
- [26] P. S. K., S. A. Kesanapalli, Y. Simmhan, Characterizing the performance of accelerated Jetson edge devices for training deep learning models, Proceedings of the ACM on Measurement and Analysis of Computing Systems 7 (1) (2023). doi:10.1145/3570604.
- [27] Q. Zhang, F. Machida, L. Carnevali, Input resolution matters: Real-time object detection latency, in: IEEE International Symposium on Modelling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS), 2026, arXiv:2609.12920.
- [28] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Zhang, A. Zhong, P. Zhou, Y. Zhu, MLPerf inference benchmark, in: ACM/IEEE International Symposium on Computer Architecture (ISCA), 2020, pp. 446–459.
- [29] W. Jang, H. Jeong, K. Kang, N. Dutt, J.-C. Kim, R-TOD: Real-time object detector with minimized end-to-end delay for autonomous driving, in: IEEE Real-Time Systems Symposium (RTSS), 2020, arXiv:2011.06372.
- [30] T.-W. Chin, R. Ding, D. Marculescu, AdaScale: Towards real-time video object detection using adaptive scaling, in: Conference on Machine Learning and Systems (SysML), 2019.
- [31] M. Zhu, K. Han, E. Wu, Q. Zhang, Y. Nie, Z. Lan, Y. Wang, Dynamic resolution network, in: Advances in Neural Information Processing Systems (NeurIPS), 2021.
- [32] NVIDIA, Jetson Linux R36.5.2 public sources, kernel linux-tegra-5.15, driver drivers/video/tegra/host/nvhost_podgov.c, https://developer.nvidia.com/downloads/embedded/14t/r36_release_v5.2/sources/public_sources.tbz2 (2025).
- [33] NVIDIA, Jetson Linux developer guide R36.5.2: Platform power and performance (Jetson Orin Nano, Orin NX and AGX Orin series), https://docs.nvidia.com/jetson/archives/r36.5.2/DeveloperGui
- de/SD/PlatformPowerAndPerformance/JetsonOrinNanoSeriesJetsonOrinNxSeriesAndJetsonAgxOrinSeries.html (2026).
- [34] Hailo, Hailo model zoo: pre-trained models and pre-compiled HEF files for Hailo-8 accelerators, https://github.com/hailo-ai/hailo_model_zoo (2023).
- [35] Microsoft, ONNX Runtime: cross-platform inference and training accelerator, https://github.com/microsoft/onnxruntime, version 1.30.0 used in this work, accessed 2026-10-05 (2026).
- [36] NVIDIA, TensorRT 10.3 developer guide, https://docs.nvidia.com/deeplearning/tensorrt/ (2024).
- [37] Python Software Foundation, Design of CPython’s garbage collector (Python developer’s guide), https://devguide.python.org/internals/garbage-collector/ (2024).
- [38] NVIDIA, CUDA runtime API: Device management (cudasetdevice-flags), https://docs.nvidia.com/cuda/cuda-runtime-api/ (2024).
- [39] R. J. Wysocki, CPU performance scaling (Linux kernel documentation, schedutil governor), https://docs.kernel.org/admin-guide/pm/cpufreq.html, living document, accessed 2026-10-05 (2026).
- [40] Ultralytics, Ultralytics YOLOv8 documentation, https://docs.ultralytics.com/models/yolov8/, accessed 2026-10-05 (2023).
- [41] D. T. Nguyen, L. P. Nguyen, V. H. Nguyen, S. V. Quang, M. R. Elara, A. V. Le, Raw measurement data: The governor is part of the model (YOLO26n edge-inference campaigns), [dataset] (2026). doi:10.5281/zenodo.2318580.